

Road Pothole Detection & Severity Analysis

Main Code - for project team review

Generated: 23 Sep 2026

Live demo: <https://pothole-detection-api-h1cp.onrender.com/dashboard>

Repository: <https://github.com/MohdJasimAli/pothole-detection>

OVERVIEW

The system detects potholes in road images and estimates their size, depth and severity to support smart road maintenance.

Pipeline (5 steps, implemented in main.py -> PotholeAnalysisPipeline):

1. DETECT - YOLOv8 object detection (or the classical CV fallback when PyTorch is unavailable). Output: bounding boxes + confidence.
2. SIZE - Convert pixel box to real-world cm using a calibrated baseline or a reference object; classify Small / Medium / Large.
3. DEPTH - MiDaS monocular depth map sampled inside each box; classify Shallow (<2cm) / Moderate (2-5cm) / Deep (>5cm).
4. SEVERITY- Weighted score 0-100 from size + depth -> Critical / High / Medium / Low with a maintenance response deadline.
5. REPORT - Annotated image (severity-coloured boxes) + JSON report with per-pothole metrics, summary counts and backend info.

Severity -> response time:

Critical (>=80): 24 hours | High (60-79): 1 week

Medium (35-59): 1 month | Low (<35): 3 months

Verified performance:

- 24 tests passed, 1 skipped (integration test requires torch)
- POST /api/analyze on a 3-pothole image: HTTP 200 in ~0.85s local, ~3.7s on the free cloud instance; all 3 potholes detected with independent size/depth/severity.

HOW TO RUN

Run locally:

```
pip install -r requirements.txt
pytest tests/ -q
python main.py --image data/samples/multi_potholes.jpg
python main.py --web # then open http://localhost:5000/dashboard
```

Deploy (already done):

Live dashboard: <https://pothole-detection-api-h1cp.onrender.com/dashboard>

Render auto-deploys on every push to main (build runs the test suite).

Train a custom YOLOv8 model:

```
pip install torch ultralytics
python train.py --create-config
python train.py --data dataset.yaml --epochs 100 --model yolov8s.pt
```

FILE INDEX

config.py - Configuration: paths, thresholds, size/depth/severity levels, API vars
main.py - MAIN PIPELINE - detect -> size -> depth -> severity -> report
src/detector.py - YOLOv8 detector + DetectionResult data structure (primary backend)
src/classical_detector.py - Classical CV detector: Otsu+adaptive threshold, morphology, contours
src/preprocessing.py - Image preprocessing: CLAHE contrast, denoise, lighting/shadow removal
src/depth_estimator.py - Depth estimation: MiDaS DPT + synthetic fallback, depth categories
src/size_estimator.py - Pixel -> cm size estimation (camera model) + Small/Medium/Large
src/severity.py - Severity scoring (0-100) + matrix classification + priority levels
src/utils.py - Helpers: logging, image IO, drawing boxes, IoU, report generation
api/app.py - Flask REST API: /api/analyze, /api/detect, /api/reports, /dashboard
train.py - YOLOv8 training script (dataset config, train, validate, export)
tests/test_detector.py - Unit tests - 25 tests covering config, preprocessing, size, severity

===== config.py | Configuration: paths, thresholds, size/depth/severity levels, API vars =====

"""

Configuration module for Road Pothole Detection and Severity Analysis System.
Contains all configuration constants, paths, and settings.
"""

import os

=====
Device Detection
=====

def _torch_usable():
 """Return True only if torch is installed AND its native libs load.

The probe runs in a subprocess because a broken torch install can raise
beyond ImportError/OSError (segfault / access violation), which would
otherwise crash this interpreter during a plain `import torch`.
"""

try:
 from importlib import metadata as _metadata
 _metadata.version("torch")

except Exception:
 return False

try:
 import subprocess as _sp
 import sys as _sys
 r = _sp.run(
 [_sys.executable, "-c",
 "import torch; "
 "assert hasattr(torch, 'cuda'); "
 "torch.cuda.is_available()"],
 capture_output=True, timeout=120,
)
 return r.returncode == 0

except Exception:
 return False

try:
 _TORCH_OK = _torch_usable()
except Exception:
 _TORCH_OK = False

if _TORCH_OK:
 import torch

 DEFAULT_DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
 TORCH_AVAILABLE = True
else:
 TORCH_AVAILABLE = False
 DEFAULT_DEVICE = "cpu"

=====
Project Paths
=====
PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))

Data directories
DATA_DIR = os.path.join(PROJECT_ROOT, "data")
MODELS_DIR = os.path.join(DATA_DIR, "models")
SAMPLES_DIR = os.path.join(DATA_DIR, "samples")
UPLOADS_DIR = os.path.join(PROJECT_ROOT, "frontend", "uploads")

Model paths
YOLO_MODEL_PATH = os.path.join(MODELS_DIR, "best.pt")
MIDAS_MODEL_PATH = os.path.join(MODELS_DIR, "midas")

Detection backend mode. When True (default), the pipeline falls back to the
classical image-processing detector (src/classical_detector.py) on machines
where PyTorch/Ultralytics cannot be installed - for example sandboxed VMs
where the torch native DLLs cannot initialize. When False, missing torch
raises ImportError instead.
ALLOW_CLASSICAL_FALLBACK = True

=====
Detection Settings
=====
YOLO detection thresholds
CONFIDENCE_THRESHOLD = 0.4 # Minimum confidence for detection

```

IOU_THRESHOLD = 0.45          # NMS threshold
IMAGE_SIZE = 640              # Input image size for YOLO

# Classes to detect (0 = pothole in our trained model)
CLASS_NAMES = {0: "Pothole"}

# =====
# Image Processing Settings
# =====
# Camera calibration parameters (for size estimation)
# These should be calibrated based on your camera setup
CAMERA_HEIGHT_CM = 80         # Height of camera from ground (cm)
FOCAL_LENGTH_PIXELS = 800     # Focal length in pixels
SENSOR_WIDTH_MM = 36          # Sensor width in mm
REFERENCE_OBJECT_SIZE_CM = 30 # Known reference (e.g., road marking width in cm)

# Baseline pixel-to-cm scale used when no reference object is supplied.
# Assumes a 640 px wide image of a pothole captured from ~1-1.5 m away,
# covering roughly 220 cm of ground (~0.35 cm per pixel).
#
# WARNING: This is an ESTIMATE ONLY. Real-world accuracy requires
# calibration with a reference object of known size. Adjust this value
# for your specific camera setup and mounting height.
PIXEL_TO_CM_BASELINE = 0.35

# Preprocessing settings
CLAHE_CLIP_LIMIT = 3.0        # Contrast enhancement clip limit
CLAHE_TILE_GRID_SIZE = (8, 8) # CLAHE tile grid size
GAUSSIAN_BLUR_KERNEL = (3, 3) # Gaussian blur kernel size

# =====
# Depth Estimation Settings
# =====
# MiDaS model settings
MIDAS_MODEL_TYPE = "DPT_Hybrid" # "DPT_Hybrid", "DPT_Large", "DPT_Small"
MIDAS_TARGET_SIZE = (384, 384) # Input size for MiDaS

# Depth range for normalization (in meters, relative)
MIN_DEPTH = 0.1
MAX_DEPTH = 10.0

# =====
# Severity Classification Thresholds
# =====
# Size thresholds in cm2
SIZE_SMALL_MAX_CM2 = 500      # < 500 cm2 -> Small
SIZE_MEDIUM_MAX_CM2 = 2000    # 500-2000 cm2 -> Medium
# > 2000 cm2 -> Large

# Depth thresholds in cm (estimated)
DEPTH_SHALLOW_MAX_CM = 2.0   # < 2 cm -> Shallow
DEPTH_MODERATE_MAX_CM = 5.0   # 2-5 cm -> Moderate
# > 5 cm -> Deep

# Upper bound used to map normalized relative-depth differences to a
# plausible physical pothole depth in cm. Calibrate against measured
# ground truth for survey-grade accuracy.
DEPTH_MAX_CM_ESTIMATE = 25.0

# Severity priority mapping
SEVERITY_LEVELS = {
    "Critical": {"color": "red", "priority": 1, "response_time_hours": 24},
    "High": {"color": "orange", "priority": 2, "response_time_hours": 168},
    "Medium": {"color": "yellow", "priority": 3, "response_time_hours": 720},
    "Low": {"color": "green", "priority": 4, "response_time_hours": 2160},
}

# Size categories
SIZE_CATEGORIES = {
    "Small": {"min": 0, "max": SIZE_SMALL_MAX_CM2},
    "Medium": {"min": SIZE_SMALL_MAX_CM2, "max": SIZE_MEDIUM_MAX_CM2},
    "Large": {"min": SIZE_MEDIUM_MAX_CM2, "max": float("inf")},
}

# Depth categories
DEPTH_CATEGORIES = {
    "Shallow": {"min": 0, "max": DEPTH_SHALLOW_MAX_CM},
    "Moderate": {"min": DEPTH_SHALLOW_MAX_CM, "max": DEPTH_MODERATE_MAX_CM},
    "Deep": {"min": DEPTH_MODERATE_MAX_CM, "max": float("inf")},
}

# =====

```

```

# API Settings
# =====
API_HOST = os.environ.get("HOST", "0.0.0.0")
# Render / Railway / Heroku inject $PORT at runtime; fall back to 5000 locally.
API_PORT = int(os.environ.get("PORT", "5000"))
# Debug must be OFF in production (never expose the Werkzeug debugger).
# Set API_DEBUG=true explicitly for local development only.
API_DEBUG = os.environ.get("API_DEBUG", "false").lower() == "true"

# Maximum upload size (MB)
MAX_CONTENT_LENGTH_MB = 50

# Allowed file extensions
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "bmp", "tiff"}

# =====
# Web Frontend Settings
# =====
WEB_HOST = "127.0.0.1"
WEB_PORT = 5000

# Mapbox / Leaflet settings
MAP_DEFAULT_LAT = 37.7749 # Default center latitude
MAP_DEFAULT_LNG = -122.4194 # Default center longitude
MAP_DEFAULT_ZOOM = 15

# =====
# Flask Secret Key (for session management)
# =====
SECRET_KEY = os.environ.get("FLASK_SECRET_KEY", "pothole-detection-secret-key-2024")

# =====
# Database Settings (SQLite for development)
# =====
DATABASE_PATH = os.path.join(PROJECT_ROOT, "data", "pothole_reports.db")
SQLALCHEMY_DATABASE_URI = f"sqlite:/// {DATABASE_PATH}"
SQLALCHEMY_TRACK_MODIFICATIONS = False

# =====
# Logging Settings
# =====
LOG_LEVEL = "INFO"
LOG_FILE = os.path.join(PROJECT_ROOT, "logs", "app.log")

# =====
# Utility Functions
# =====
def ensure_directories():
    """Create all required directories if they don't exist."""
    for d in [DATA_DIR, MODELS_DIR, SAMPLES_DIR, UPLOADS_DIR, os.path.dirname(LOG_FILE)]:
        os.makedirs(d, exist_ok=True)

# Ensure directories exist on import
ensure_directories()

```

```
=====
main.py | MAIN PIPELINE - detect -> size -> depth -> severity -> report
=====
```

```
"""
```

```
Main Pipeline Orchestrator
```

```
=====
```

```
Ties together detection, depth estimation, size estimation,
and severity classification into a single pipeline.
```

```
Usage:
```

```
python main.py --image road.jpg
python main.py --batch data/samples/
python main.py --web
```

```
"""
```

```
import os
import sys
import argparse
import json
import cv2
import numpy as np
from typing import List, Dict, Tuple, Optional
from datetime import datetime
from glob import glob

import config
from src.detector import PotholeDetector, DetectionResult
from src.depth_estimator import DepthEstimator, DepthResult
from src.size_estimator import SizeEstimator, SizeResult
from src.severity import SeverityClassifier, SeverityResult
from src.utils import setup_logger, save_json, load_image, draw_bounding_boxes
```

```
logger = setup_logger("main", config.LOG_FILE)
```

```
class PotholeAnalysisPipeline:
```

```
    """
```

```
    Complete pipeline: detect -> estimate size -> estimate depth -> classify severity.
```

```
    This orchestrator combines all sub-modules into a single end-to-end system.
```

```
    """
```

```
    def __init__(self,
                  detector: PotholeDetector = None,
                  depth_estimator: DepthEstimator = None,
                  size_estimator: SizeEstimator = None,
                  severity_classifier: SeverityClassifier = None,
                  device: str = None):
```

```
        """
```

```
        Initialize the analysis pipeline.
```

```
        Args:
```

```
        detector: PotholeDetector instance (creates default if None)
        depth_estimator: DepthEstimator instance
        size_estimator: SizeEstimator instance
        severity_classifier: SeverityClassifier instance
        device: Device for inference
```

```
        """
```

```
        self.device = device or config.DEFAULT_DEVICE
        self.size_estimator = size_estimator or SizeEstimator()
        self.severity_classifier = severity_classifier or SeverityClassifier()
```

```
        # Detector: prefer YOLOv8, but fall back to the classical detector
        # when PyTorch/Ultralytics are unavailable (see config.ALLOW_CLASSICAL_FALLBACK).
        self.detection_backend = "yolo"
```

```
        if detector is not None:
            self.detector = detector
```

```
        else:
```

```
            try:
                self.detector = PotholeDetector(device=self.device)
            except Exception as e:
                if not getattr(config, "ALLOW_CLASSICAL_FALLBACK", True):
                    raise
                from src.classical_detector import ClassicalPotholeDetector
                logger.warning(
                    f"YOLOv8 unavailable ({e}); "
                    f"using ClassicalPotholeDetector fallback."
                )
                self.detector = ClassicalPotholeDetector()
                self.detection_backend = "classical"
```

```
        # Depth: prefer MiDaS, but synthesized brightness-based depth is used
```

```

# inside DepthEstimator when its torch model cannot load.
if depth_estimator is not None:
    self.depth_estimator = depth_estimator
else:
    try:
        self.depth_estimator = DepthEstimator(device=self.device)
    except Exception as e:
        logger.warning(
            f"MiDaS unavailable ({e}); depth will use the "
            f"synthetic fallback inside DepthEstimator."
        )
        self.depth_estimator = DepthEstimator.__new__(DepthEstimator)
        self.depth_estimator.model = None
        self.depth_estimator.model_type = "synthetic-fallback"
        self.depth_estimator.device = self.device

self.depth_backend = getattr(self.depth_estimator, "model_type", "unknown")

logger.info(
    f"PotholeAnalysisPipeline initialized "
    f"(detector={self.detection_backend}, depth={self.depth_backend})"
)

def analyze_image(self, image: np.ndarray,
                  gps_coords: Tuple[float, float] = None,
                  reference_bbox: Tuple[int, int, int, int] = None,
                  visualize: bool = True) -> Dict:
    """
    Run the complete analysis pipeline on a single image.

    Args:
        image: Input road image (BGR format)
        gps_coords: Optional GPS coordinates (lat, lng)
        reference_bbox: Optional reference object bbox for calibration
        visualize: Whether to draw results on the image

    Returns:
        Dictionary containing:
            - detections: List of enriched detection results
            - annotated_image: Image with visualizations (if visualize=True)
            - report: Complete analysis report
    """
    start_time = datetime.now()
    img_h, img_w = image.shape[:2]

    # Step 1: Detect potholes
    logger.info("Step 1: Running pothole detection...")
    results = self.detector.detect(image)
    raw_detections = self.detector.extract_detections(results)
    logger.info(f" Found {len(raw_detections)} pothole(s)")

    # Step 2: Estimate size for each detection
    logger.info("Step 2: Estimating pothole sizes...")
    for det in raw_detections:
        size_result = self.size_estimator.estimate_size(
            det.bbox, img_w, reference_bbox
        )
        det.width_cm = size_result.width_cm
        det.height_cm = size_result.height_cm
        det.size_cm2 = size_result.area_cm2
        det.size_category = size_result.size_category

    # Step 3: Estimate depth for each detection
    logger.info("Step 3: Estimating pothole depths...")
    depth_map = self.depth_estimator.estimate_depth(image)

    for det in raw_detections:
        depth_cm, depth_cat = self.depth_estimator.get_pothole_depth(
            depth_map, det.bbox
        )
        det.depth_cm = depth_cm
        det.depth_category = depth_cat

    # Step 4: Classify severity
    logger.info("Step 4: Classifying severity...")
    for det in raw_detections:
        severity_result = self.severity_classifier.classify(
            area_cm2=det.size_cm2,
            size_category=det.size_category,
            depth_cm=det.depth_cm,
            depth_category=det.depth_category,
        )

```

```

        det.severity = severity_result.severity
        det.priority = severity_result.priority
        det.response_time_hours = severity_result.response_time_hours

# Step 5: Visualize
annotated_image = None
if visualize:
    annotated_image = self._annotate_image(image, raw_detections, depth_map)

elapsed = (datetime.now() - start_time).total_seconds()

# Generate report
report = self._generate_report(
    raw_detections, image, gps_coords, elapsed
)

logger.info(f"Pipeline complete in {elapsed:.2f}s")

return {
    "detections": [d.to_dict() for d in raw_detections],
    "annotated_image": annotated_image,
    "report": report,
    "processing_time_seconds": elapsed,
}

def _annotate_image(self, image: np.ndarray,
                    detections: List[DetectionResult],
                    depth_map: np.ndarray) -> np.ndarray:
    """Create annotated output image with boxes, labels, and severity colors."""
    from src.utils import draw_bounding_boxes as original_draw
    # Use custom drawing with severity colors
    img = image.copy()

    for det in detections:
        x1, y1, x2, y2 = det.bbox
        color = det.get_color() if hasattr(det, 'get_color') else (0, 255, 0)

        # Use severity color
        from src.severity import SeverityClassifier
        temp_result = self.severity_classifier.classify(
            area_cm2=det.size_cm2,
            size_category=det.size_category,
            depth_cm=det.depth_cm,
            depth_category=det.depth_category,
        )
        color = temp_result.color_bgr

        # Draw bounding box
        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)

        # Draw label
        label = f"Pothole: {det.confidence:.2f} | "
        label += f"{det.size_cm2:.0f}cm2 ({det.size_category}) | "
        label += f"D:{det.depth_cm:.1f}cm ({det.depth_category}) | "
        label += f"{det.severity}"

        (w, h), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.4, 1)
        cv2.rectangle(img, (x1, y1 - h - 4), (x1 + w, y1), color, -1)
        cv2.putText(img, label, (x1, y1 - 2),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 255, 255), 1)

    return img

def _generate_report(self, detections: List[DetectionResult],
                    image: np.ndarray,
                    gps_coords: Tuple[float, float],
                    elapsed: float) -> Dict:
    """Generate a complete analysis report."""
    detection_dicts = [d.to_dict() for d in detections]

    critical = sum(1 for d in detections if d.severity == "Critical")
    high = sum(1 for d in detections if d.severity == "High")
    medium = sum(1 for d in detections if d.severity == "Medium")
    low = sum(1 for d in detections if d.severity == "Low")

    report = {
        "timestamp": datetime.now().isoformat(),
        "gps_coordinates": {"lat": gps_coords[0], "lng": gps_coords[1]} if gps_coords else None,
        "image_shape": {"height": image.shape[0], "width": image.shape[1]},
        "total_potholes": len(detections),
        "severity_summary": {
            "critical": critical,

```

```

        "high": high,
        "medium": medium,
        "low": low,
    },
    "potholes": detection_dicts,
    "processing_time_seconds": round(elapsed, 3),
    "model_info": {
        "detector": getattr(self, "detection_backend", "yolo"),
        "depth_model": getattr(self, "depth_backend",
                                "MiDaS DPT-Hybrid"),
        "device": self.device,
    },
}
return report

def analyze_from_path(self, image_path: str,
                      gps_coords: Tuple[float, float] = None,
                      output_dir: str = None) -> Dict:
    """Analyze an image from file path and optionally save results."""
    image = load_image(image_path)
    result = self.analyze_image(image, gps_coords=gps_coords)

    if output_dir:
        os.makedirs(output_dir, exist_ok=True)
        filename = os.path.splitext(os.path.basename(image_path))[0]

        # Save annotated image
        annotated_path = os.path.join(output_dir, f"{filename}_annotated.jpg")
        cv2.imwrite(annotated_path, result["annotated_image"])

        # Save report
        report_path = os.path.join(output_dir, f"{filename}_report.json")
        save_json(result["report"], report_path)

        result["annotated_image_path"] = annotated_path
        result["report_path"] = report_path

    return result

def process_folder(self, folder_path: str,
                  gps_coords: Tuple[float, float] = None,
                  output_dir: str = None,
                  pattern: str = ".*") -> List[Dict]:
    """Process all images in a folder."""
    image_extensions = {".jpg", ".jpeg", ".png", ".bmp", ".tiff", ".gif"}
    files = glob(os.path.join(folder_path, "**", pattern), recursive=True)
    image_files = [f for f in files
                    if os.path.splitext(f)[1].lower() in image_extensions]

    results = []
    for image_path in image_files:
        logger.info(f"Processing: {image_path}")
        try:
            result = self.analyze_from_path(image_path, gps_coords, output_dir)
            results.append(result)
        except Exception as e:
            logger.error(f"Failed to process {image_path}: {e}")

    return results

def main():
    """Command-line entry point."""
    parser = argparse.ArgumentParser(
        description="Road Pothole Detection and Severity Analysis"
    )
    parser.add_argument("--image", type=str, default=None,
                        help="Path to input image")
    parser.add_argument("--batch", type=str, default=None,
                        help="Path to folder for batch processing")
    parser.add_argument("--output", type=str, default="outputs",
                        help="Output directory for results")
    parser.add_argument("--gps-lat", type=float, default=None,
                        help="GPS latitude")
    parser.add_argument("--gps-lng", type=float, default=None,
                        help="GPS longitude")
    parser.add_argument("--web", action="store_true",
                        help="Start web API server")

    args = parser.parse_args()

    if args.web:

```

```

# Start Flask web server
from api.app import app
app.run(host=config.API_HOST, port=config.API_PORT, debug=config.API_DEBUG)
return

# Initialize pipeline
logger.info("Initializing Pothole Analysis Pipeline...")
pipeline = PotholeAnalysisPipeline()

gps = None
if args.gps_lat and args.gps_lng:
    gps = (args.gps_lat, args.gps_lng)

if args.image:
    # Single image processing
    result = pipeline.analyze_from_path(args.image, gps, args.output)
    print(f"\n{' '*60}")
    print(f"Results for: {args.image}")
    print(f"{' '*60}")
    for det in result["detections"]:
        print(f"  Pothole: {det['confidence']:.2f} conf")
        print(f"  Size: {det['size_cm2']} cm2 ({det['size_category']})")
        print(f"  Depth: {det['depth_cm']} cm ({det['depth_category']})")
        print(f"  Severity: {det['severity']} (Priority {det['priority']})")
        print(f"  Response: {det['response_time_hours']}h")
    print(f"\nProcessing time: {result['processing_time_seconds']:.2f}s")
    print(f"Annotated image: {result.get('annotated_image_path', 'N/A')}")
    print(f"Report: {result.get('report_path', 'N/A')}")

elif args.batch:
    results = pipeline.process_folder(args.batch, gps, args.output)
    print(f"\nProcessed {len(results)} images successfully.")

else:
    print("No input specified. Use --image or --batch.")
    parser.print_help()

if __name__ == "__main__":
    main()

```

=====

src/detector.py | YOLOv8 detector + DetectionResult data structure (primary backend)

=====

"""

Pothole Detection Module

=====

Uses YOLOv8 (Ultralytics) for real-time pothole detection from road images.

Usage:

```
from src.detector import PotholeDetector

detector = PotholeDetector(model_path="data/models/best.pt")
results = detector.detect(image)
detections = detector.extract_detections(results)
```

"""

```
import os
import cv2
import numpy as np
from typing import List, Dict, Tuple, Optional, Any
from dataclasses import dataclass
```

def _torch_usable():

"""Return True only if torch is installed AND its native libs load.

The probe runs in a subprocess because a broken torch install can raise beyond ImportError/OSError (segfault / access violation), which would otherwise crash this interpreter during a plain `import torch`.

"""

try:

```
    from importlib import metadata as _metadata
    _metadata.version("torch")
```

except Exception:

return False

try:

```
    import subprocess as _sp
    import sys as _sys
    r = _sp.run(
        [_sys.executable, "-c",
         "import torch; "
         "assert hasattr(torch, 'cuda'); "
         "torch.cuda.is_available()"],
        capture_output=True, timeout=120,
    )
    return r.returncode == 0
```

except Exception:

return False

try:

```
    _TORCH_OK = _torch_usable()
```

except Exception:

```
    _TORCH_OK = False
```

if _TORCH_OK:

```
    import torch
    from ultralytics import YOLO
    TORCH_AVAILABLE = True
```

else:

```
    TORCH_AVAILABLE = False
```

import config

from . import utils

from . import preprocessing

from .utils import setup_logger

logger = setup_logger(__name__)

@dataclass

class DetectionResult:

"""Data class representing a single pothole detection."""

```
bbox: List[int]
confidence: float
class_id: int
class_name: str
width_px: int
height_px: int
center: Tuple[int, int]
size_cm2: float = 0.0
size_category: str = ""
```

```

width_cm: float = 0.0
height_cm: float = 0.0
depth_cm: float = 0.0
depth_category: str = ""
severity: str = ""
priority: int = 4
response_time_hours: int = 0

def to_dict(self) -> Dict[str, Any]:
    return {
        "bbox": self.bbox,
        "confidence": round(self.confidence, 4),
        "class_id": self.class_id,
        "class_name": self.class_name,
        "width_px": self.width_px,
        "height_px": self.height_px,
        "center": list(self.center),
        "size_cm2": round(self.size_cm2, 2),
        "size_category": self.size_category,
        "width_cm": round(self.width_cm, 2),
        "height_cm": round(self.height_cm, 2),
        "depth_cm": round(self.depth_cm, 2),
        "depth_category": self.depth_category,
        "severity": self.severity,
        "priority": self.priority,
        "response_time_hours": self.response_time_hours,
    }

```

```

class PotholeDetector:
    """Pothole detection using YOLOv8."""

    def __init__(self, model_path=None, device=None,
                 conf_thres=None, iou_thres=None):
        if not TORCH_AVAILABLE:
            raise ImportError(
                "PyTorch and Ultralytics YOLO are required. "
                "Install with: pip install torch ultralytics"
            )
        if model_path is None:
            model_path = config.YOLO_MODEL_PATH
        self.conf_thres = conf_thres or config.CONFIDENCE_THRESHOLD
        self.iou_thres = iou_thres or config.IOU_THRESHOLD
        self.device = device or config.DEFAULT_DEVICE
        self.model = self._load_model(model_path)
        self.model.to(self.device)
        logger.info(f"PotholeDetector initialized on device: {self.device}")

    def _load_model(self, model_path):
        if os.path.exists(model_path):
            logger.info(f"Loading custom model from: {model_path}")
            return YOLO(model_path)
        for name in ["yolov8n.pt", "yolov8s.pt", "yolov8m.pt"]:
            try:
                logger.info(f"Trying model: {name}")
                return YOLO(name)
            except Exception:
                continue
        raise FileNotFoundError(
            f"Model not found at {model_path}. "
            "Please download a YOLOv8 model or provide a valid model path."
        )

    def detect(self, image, preprocess_input=False):
        if preprocess_input:
            img = preprocessing.enhance_contrast(image)
        else:
            img = image
        results = self.model(
            img, conf=self.conf_thres, iou=self.iou_thres,
            verbose=False, device=self.device,
        )
        return results[0] if len(results) > 0 else results

    def detect_and_extract(self, image):
        results = self.detect(image)
        return self.extract_detections(results)

    def detect_from_path(self, image_path):
        image = utils.load_image(image_path)
        detections = self.detect_and_extract(image)
        return image, detections

```

```

def extract_detections(self, results):
    """Extract structured detection results from YOLO output."""
    detections = []
    if results.bboxes is None or len(results.bboxes) == 0:
        return detections
    if hasattr(results, 'orig_shape'):
        image_shape = results.orig_shape
    else:
        image_shape = (720, 1280)
    img_h, img_w = image_shape[:2]
    for box in results.bboxes:
        xyxy = box.xyxy[0].cpu().numpy()
        conf = float(box.conf.cpu().numpy()[0])
        cls_id = int(box.cls.cpu().numpy()[0])
        x1, y1, x2, y2 = xyxy
        x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
        x1, y1 = max(0, x1), max(0, y1)
        x2, y2 = min(img_w - 1, x2), min(img_h - 1, y2)
        width_px = x2 - x1
        height_px = y2 - y1
        class_name = config.CLASS_NAMES.get(cls_id, f"class_{cls_id}")
        detection = DetectionResult(
            bbox=[x1, y1, x2, y2], confidence=conf, class_id=cls_id,
            class_name=class_name, width_px=width_px,
            height_px=height_px,
            center=((x1 + x2) // 2, (y1 + y2) // 2),
        )
        detections.append(detection)
    logger.info(f"Extracted {len(detections)} detections")
    return detections

def visualize_detections(self, image, detections, show_labels=True):
    """Draw detection results on the image."""
    img = image.copy()
    for det in detections:
        x1, y1, x2, y2 = det.bbox
        color = (0, 255, 0)
        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
        if show_labels:
            label = f"Pothole: {det.confidence:.2f}"
            (w, h), _ = cv2.getTextSize(
                label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
            cv2.rectangle(img, (x1, y1 - h - 4),
                          (x1 + w, y1), color, -1)
            cv2.putText(img, label, (x1, y1 - 2),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                        (0, 0, 0), 1)
    return img

def get_model_info(self):
    return {
        "model_path": config.YOLO_MODEL_PATH,
        "device": self.device,
        "conf_threshold": self.conf_thres,
        "iou_threshold": self.iou_thres,
        "class_names": config.CLASS_NAMES,
    }

```

=====

src/classical_detector.py | Classical CV detector: Otsu+adaptive threshold, morphology, contours

=====

"""

Classical Computer-Vision Pothole Detector (no deep-learning dependency)

=====

Fallback detector using traditional image processing:

1. Grayscale + CLAHE contrast enhancement
2. GLOBAL (Otsu) + LOCAL (adaptive) thresholding combined - Otsu catches the full body of large dark blobs, adaptive catches edges/subtle regions; neither alone is sufficient
3. Brightness verification against a LARGE-window local mean (window scales with image size) so big uniform potholes are not rejected
4. Morphological open/close/dilate clean-up
5. Contour extraction with shape/area filtering
6. Nearby-box merging so one pothole fragmented into several contours reports as a single bounding box

Usage:

```
from src.classical_detector import ClassicalPotholeDetector
detector = ClassicalPotholeDetector()
detections = detector.detect_and_extract(image)
```

"""

```
import cv2
import numpy as np
from typing import List, Tuple, Optional
```

```
import config
from .utils import setup_logger
```

```
logger = setup_logger(__name__)
```

```
class ClassicalPotholeDetector:
```

"""

Pothole detector using classical image processing.

Detects potholes as dark, roughly elliptical regions on the road surface.
Suitable as a CPU-only, dependency-light fallback to YOLOv8.

Attributes:

min_area_px: Minimum contour area (pixels) to count as a pothole
max_area_ratio: Maximum contour area as a fraction of the image
darkness_threshold: How much darker than the local mean a region must be
merge_gap_px: Max pixel gap between fragments merged into one pothole

"""

```
def __init__(self,
              min_area_px: int = 300,
              max_area_ratio: float = 0.30,
              darkness_threshold: float = 0.80,
              block_size: int = 51,
              constant_c: int = 10,
              merge_gap_px: int = 40):
    self.min_area_px = min_area_px
    self.max_area_ratio = max_area_ratio
    self.darkness_threshold = darkness_threshold
    self.block_size = block_size if block_size % 2 == 1 else block_size + 1
    self.constant_c = constant_c
    self.merge_gap_px = merge_gap_px
    logger.info(
        f"ClassicalPotholeDetector initialized "
        f"(min_area={min_area_px}px, block={self.block_size}, "
        f"C={constant_c}, merge_gap={merge_gap_px}px)"
    )
```

```
def get_model_info(self) -> dict:
    """Return detector information (mirrors PotholeDetector API)."""
    return {
        "model_path": "classical-cv (no weights)",
        "device": "cpu",
        "conf_threshold": self.darkness_threshold,
        "iou_threshold": None,
        "class_names": config.CLASS_NAMES,
        "method": "otsu+adaptive threshold + morphology + contours",
    }
```

```
def detect(self, image: np.ndarray, preprocess_input: bool = False):
    """
    Run classical detection.
```

```

Args:
    image: BGR image
    preprocess_input: Accepted for API compatibility (unused)

Returns:
    List of (contour_or_None, bbox, score, area) tuples
"""
return self._find_pothole_contours(image)

def _find_pothole_contours(self, image: np.ndarray):
    """Core classical detection routine."""
    img_h, img_w = image.shape[:2]
    img_area = float(img_h * img_w)

    # 1. Grayscale + contrast enhancement
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=config.CLAHE_CLIP_LIMIT,
                             tileGridSize=config.CLAHE_TILE_GRID_SIZE)
    gray = clahe.apply(gray)

    # 2. Mild blur to suppress texture noise
    blurred = cv2.GaussianBlur(gray, (7, 7), 0)

    # 3a. GLOBAL threshold (Otsu): captures the FULL body of large
    #      uniform dark blobs, which adaptive thresholding alone misses
    #      (inside a blob bigger than the window, the local neighborhood
    #      is equally dark so nothing looks "darker than local mean").
    _, otsu_mask = cv2.threshold(
        blurred, 0, 255,
        cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # 3b. LOCAL adaptive threshold: catches edges and subtle regions
    adaptive_mask = cv2.adaptiveThreshold(
        blurred, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        self.block_size,
        self.constant_c,
    )

    combined = cv2.bitwise_or(otsu_mask, adaptive_mask)

    # 4. Morphological clean-up + slight dilation to bridge gaps
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
    combined = cv2.morphologyEx(combined, cv2.MORPH_OPEN, kernel,
                                iterations=2)
    combined = cv2.morphologyEx(combined, cv2.MORPH_CLOSE, kernel,
                                iterations=3)

    # 5. Brightness verification against a LARGE-window local mean.
    #      The window must be much bigger than the biggest expected
    #      pothole (~1/3 of image dims) so the interior of a large blob
    #      is still "darker than its surroundings".
    win = max(151, (max(img_h, img_w) // 3) | 1) # force odd
    local_mean = cv2.blur(blurred.astype(np.float32), (win, win))
    brightness_ok = (blurred.astype(np.float32) <
                     local_mean * self.darkness_threshold)
    mask = cv2.bitwise_and(combined,
                           (brightness_ok * 255).astype(np.uint8))

    # 6. Extract contours
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)

    results = []
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < self.min_area_px or area > img_area * self.max_area_ratio:
            continue

        x, y, w, h = cv2.boundingRect(cnt)
        if h == 0 or w == 0:
            continue

        # Shape filters: potholes are roughly elliptical, not thin lines
        aspect = w / float(h)
        if aspect < 0.25 or aspect > 4.0:
            continue

        # Circularity: 4*pi*area / perimeter^2
        perimeter = cv2.arcLength(cnt, True)

```

```

        if perimeter <= 0:
            continue
        circularity = 4 * np.pi * area / (perimeter * perimeter)
        if circularity < 0.15:
            continue

        # Fill ratio: how much of the bbox the blob occupies
        fill_ratio = area / float(w * h)
        if fill_ratio < 0.30:
            continue

        # Confidence: combination of circularity, fill and darkness
        region = gray[y:y + h, x:x + w]
        local_ref = local_mean[y:y + h, x:x + w]
        darkness = 1.0 - (float(np.mean(region)) /
                          max(float(np.mean(local_ref)), 1.0))
        darkness = float(np.clip(darkness * 3.0, 0.0, 1.0))

        score = float(np.clip(
            0.4 * min(circularity / 0.8, 1.0) +
            0.3 * min(fill_ratio / 0.7, 1.0) +
            0.3 * darkness,
            0.05, 0.99,
        ))

        results.append((cnt, [x, y, x + w, y + h], score, area))

# 7. Merge nearby fragments into single pothole boxes
results = self._merge_results(results, self.merge_gap_px)
return results

@staticmethod
def _merge_results(results, gap_px: int = 40):
    """
    Merge detection fragments whose bounding boxes are within `gap_px`
    of each other into a single detection (union box, max score,
    summed area). A single pothole often fragments into several
    contours after thresholding.
    """
    if not results:
        return results
    boxes = [list(r[1]) for r in results]
    scores = [r[2] for r in results]
    areas = [r[3] for r in results]
    used = [False] * len(boxes)
    merged = []
    for i in range(len(boxes)):
        if used[i]:
            continue
        x1, y1, x2, y2 = boxes[i]
        best = scores[i]
        tot = areas[i]
        used[i] = True
        changed = True
        while changed:
            changed = False
            for j in range(len(boxes)):
                if used[j]:
                    continue
                bx1, by1, bx2, by2 = boxes[j]
                if not (bx1 - gap_px > x2 or bx2 + gap_px < x1 or
                        by1 - gap_px > y2 or by2 + gap_px < y1):
                    x1 = min(x1, bx1)
                    y1 = min(y1, by1)
                    x2 = max(x2, bx2)
                    y2 = max(y2, by2)
                    best = max(best, scores[j])
                    tot += areas[j]
                    used[j] = True
                    changed = True
            merged.append((None, [x1, y1, x2, y2], best, tot))
    merged.sort(key=lambda r: r[3], reverse=True)
    return merged

def extract_detections(self, classical_results):
    """Convert classical results into DetectionResult objects."""
    from .detector import DetectionResult

    detections: List = []
    for _cnt, bbox, score, _area in classical_results:
        x1, y1, x2, y2 = bbox
        detections.append(DetectionResult(

```

```

        bbox=[x1, y1, x2, y2],
        confidence=round(score, 4),
        class_id=0,
        class_name=config.CLASS_NAMES.get(0, "Pothole"),
        width_px=x2 - x1,
        height_px=y2 - y1,
        center=((x1 + x2) // 2, (y1 + y2) // 2),
    ))
    logger.info(f"Classical detector found {len(detections)} candidate(s)")
    return detections

def detect_and_extract(self, image: np.ndarray):
    """Convenience: detect + convert in one step."""
    return self.extract_detections(self.detect(image))

def detect_from_path(self, image_path: str):
    """Load image from disk, detect, and return (image, detections)."""
    from .utils import load_image
    image = load_image(image_path)
    return image, self.detect_and_extract(image)

def visualize_detections(self, image: np.ndarray, detections,
                        show_labels: bool = True) -> np.ndarray:
    """Draw detections (mirrors PotholeDetector.visualize_detections)."""
    img = image.copy()
    for det in detections:
        x1, y1, x2, y2 = det.bbox
        color = (0, 255, 0)
        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
        if show_labels:
            label = f"Pothole: {det.confidence:.2f}"
            (w, h), _ = cv2.getTextSize(label,
                                       cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
            cv2.rectangle(img, (x1, y1 - h - 4), (x1 + w, y1), color, -1)
            cv2.putText(img, label, (x1, y1 - 2),
                       cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1)
    return img

```

```
=====
src/preprocessing.py | Image preprocessing: CLAHE contrast, denoise, lighting/shadow removal
=====
```

```
"""
```

```
Image Preprocessing Module
```

```
=====
```

```
Handles image enhancement, normalization, and preparation
for deep learning models.
```

```
"""
```

```
import cv2
import numpy as np
from typing import Tuple, Optional
```

```
import config
```

```
def enhance_contrast(image: np.ndarray,
                     clip_limit: float = None,
                     tile_grid_size: Tuple[int, int] = None) -> np.ndarray:
```

```
    """
```

```
    Enhance image contrast using CLAHE.
```

```
    Args:
```

```
        image: Input image (BGR format)
        clip_limit: CLAHE clip limit
        tile_grid_size: Tile grid size for CLAHE
```

```
    Returns:
```

```
        Contrast-enhanced image
```

```
    """
```

```
    if clip_limit is None:
        clip_limit = config.CLAHE_CLIP_LIMIT
    if tile_grid_size is None:
        tile_grid_size = config.CLAHE_TILE_GRID_SIZE
```

```
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l_channel, a, b = cv2.split(lab)
```

```
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)
    l_channel = clahe.apply(l_channel)
```

```
    limg = cv2.merge((l_channel, a, b))
    enhanced = cv2.cvtColor(limg, cv2.COLOR_LAB2BGR)
```

```
    return enhanced
```

```
def reduce_noise(image: np.ndarray,
                 blur_kernel: Tuple[int, int] = None,
                 bilateral_iterations: int = 1) -> np.ndarray:
    """Reduce image noise using Gaussian blur and bilateral filtering."""
    if blur_kernel is None:
        blur_kernel = config.GAUSSIAN_BLUR_KERNEL
```

```
    denoised = cv2.GaussianBlur(image, blur_kernel, 0)
```

```
    for _ in range(bilateral_iterations):
        denoised = cv2.bilateralFilter(denoised, 9, 75, 75)
```

```
    return denoised
```

```
def normalize_image(image: np.ndarray,
                    mean: Tuple[float, float, float] = (0.485, 0.456, 0.406),
                    std: Tuple[float, float, float] = (0.229, 0.224, 0.225)) -> np.ndarray:
```

```
    """
```

```
    Normalize image pixel values using ImageNet statistics.
```

```
    Args:
```

```
        image: Input image (uint8)
        mean: Per-channel mean values
        std: Per-channel standard deviation values
```

```
    Returns:
```

```
        Normalized image (float32)
```

```
    """
```

```
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image_normalized = image_rgb.astype(np.float32) / 255.0
```

```
    for i in range(3):
        image_normalized[:, :, i] = (image_normalized[:, :, i] - mean[i]) / std[i]
```

```

    return image_normalized

def correct_lighting(image: np.ndarray) -> np.ndarray:
    """Correct non-uniform lighting using background estimation."""
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (101, 101))
    bg = cv2.morphologyEx(gray, cv2.MORPH_OPEN, kernel)
    bg = cv2.GaussianBlur(bg, (101, 101), 0)
    bg = np.maximum(bg, 1)

    result = image.copy().astype(np.float32)
    bg_3ch = np.stack([bg, bg, bg], axis=2)
    result = result / bg_3ch * 128.0
    result = np.clip(result, 0, 255).astype(np.uint8)

    return result

def remove_shadow(image: np.ndarray) -> np.ndarray:
    """Reduce shadow effects in the image."""
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    v = clahe.apply(v)
    hsv = cv2.merge([h, s, v])
    return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

def preprocess_image(image: np.ndarray,
                     normalize: bool = True,
                     target_size: Tuple[int, int] = None,
                     enhance: bool = True,
                     denoise: bool = True,
                     correct_light: bool = True) -> np.ndarray:
    """
    Full preprocessing pipeline for road images.

    Args:
        image: Input image (BGR format)
        normalize: Whether to normalize pixel values
        target_size: Optional (width, height) to resize to
        enhance: Apply CLAHE contrast enhancement
        denoise: Apply noise reduction
        correct_light: Apply lighting correction

    Returns:
        Preprocessed image
    """
    img = image.copy()

    if correct_light:
        img = correct_lighting(img)

    if enhance:
        img = enhance_contrast(img)

    if denoise:
        img = reduce_noise(img, bilateral_iterations=1)

    img = remove_shadow(img)

    if target_size is not None:
        img = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)

    if normalize:
        img = normalize_image(img)

    return img

def extract_pothole_region(image: np.ndarray,
                           bbox: Tuple[int, int, int, int],
                           padding: int = 10) -> np.ndarray:
    """
    Extract and preprocess a pothole region from the image.

    Args:
        image: Full road image
        bbox: (x1, y1, x2, y2) bounding box coordinates
    """

```

padding: Extra pixels around the bounding box

Returns:

""" Cropped and preprocessed pothole region
"""

```
x1, y1, x2, y2 = bbox  
h, w = image.shape[:2]
```

```
x1 = max(0, x1 - padding)  
y1 = max(0, y1 - padding)  
x2 = min(w, x2 + padding)  
y2 = min(h, y2 + padding)
```

```
region = image[y1:y2, x1:x2]  
region = enhance_contrast(region)  
region = reduce_noise(region, bilateral_iterations=1)
```

```
return region
```

src/depth_estimator.py | Depth estimation: MiDaS DPT + synthetic fallback, depth categories

"""

Depth Estimation Module

=====

Uses MiDaS (via PyTorch Hub) for monocular depth estimation
from pothole images.

Classes:

- DepthEstimator: Main depth estimation class

Usage:

```
from src.depth_estimator import DepthEstimator

estimator = DepthEstimator()
depth_map = estimator.estimate_depth(image)
pothole_depth = estimator.get_pothole_depth(depth_map, bbox)
```

"""

```
import os
import cv2
import numpy as np
from typing import Tuple, Optional
from dataclasses import dataclass
```

def _torch_usable():

"""Return True only if torch is installed AND its native libs load.

The probe runs in a subprocess because a broken torch install can raise
beyond ImportError/OSError (segfault / access violation), which would
otherwise crash this interpreter during a plain `import torch`.

"""

try:

```
    from importlib import metadata as _metadata
    _metadata.version("torch")
```

except Exception:

return False

try:

```
    import subprocess as _sp
    import sys as _sys
    r = _sp.run(
        [_sys.executable, "-c",
         "import torch; "
         "assert hasattr(torch, 'cuda'); "
         "torch.cuda.is_available()"],
        capture_output=True, timeout=120,
    )
    return r.returncode == 0
```

except Exception:

return False

try:

```
    _TORCH_OK = _torch_usable()
```

except Exception:

```
    _TORCH_OK = False
```

if _TORCH_OK:

```
    import torch
    import torch.nn.functional as F
    TORCH_AVAILABLE = True
```

else:

```
    TORCH_AVAILABLE = False
```

import config

from .utils import setup_logger

logger = setup_logger(__name__)

@dataclass

class DepthResult:

"""Container for depth estimation results."""

depth_map: np.ndarray # Full image depth map (H, W)

relative_depth: np.ndarray # Normalized depth [0, 1]

pothole_depth_cm: float = 0.0

depth_category: str = ""

depth_percentile: float = 0.0

```

class DepthEstimator:
    """
    Monocular depth estimation using MiDaS.

    Attributes:
        model: MiDaS model instance
        device: Device for inference
        model_type: MiDaS model variant
    """

    def __init__(self, model_type: str = None, device: str = None):
        if not TORCH_AVAILABLE:
            raise ImportError(
                "PyTorch is required for depth estimation. "
                "Install with: pip install torch"
            )

        self.model_type = model_type or config.MIDAS_MODEL_TYPE
        self.device = device or config.DEFAULT_DEVICE

        if self.model_type == "DPT_Hybrid":
            model_repo = "isl-org/MiDaS"
            model_name = "DPT_Hybrid"
        elif self.model_type == "DPT_Large":
            model_repo = "isl-org/MiDaS"
            model_name = "DPT_Large"
        else:
            model_repo = "isl-org/MiDaS"
            model_name = "DPT_Hybrid"

        self._load_model(model_repo, model_name)

        logger.info(f"DepthEstimator initialized with {self.model_type} on {self.device}")

    def _load_model(self, repo: str, model_name: str):
        """Load MiDaS model from PyTorch Hub or cache."""
        try:
            self.model = torch.hub.load(repo, model_name, pretrained=True)
            self.model.eval()
            self.model.to(self.device)

            # Load transforms
            self.transforms = torch.hub.load(repo, "transforms")[0]

            logger.info(f"MiDaS {model_name} loaded successfully")
        except Exception as e:
            logger.error(f"Failed to load MiDaS: {e}")
            logger.info("Will use synthetic depth estimation as fallback")
            self.model = None

    def estimate_depth(self, image: np.ndarray) -> np.ndarray:
        """
        Estimate depth map for an input image.

        Args:
            image: Input image (BGR format)

        Returns:
            Depth map (H, W) with normalized depth values
        """
        if self.model is None:
            return self._synthetic_depth(image)

        # Convert BGR to RGB
        img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Apply MiDaS transforms
        input_batch = self.transforms(img_rgb).unsqueeze(0).to(self.device)

        with torch.no_grad():
            depth = self.model(input_batch)

        # Resize to original image size
        depth = F.interpolate(
            depth.unsqueeze(1),
            size=img_rgb.shape[:2],
            mode="bicubic",
            align_corners=False,
        ).squeeze()

        depth_map = depth.cpu().numpy()
        return depth_map

```

```

def _synthetic_depth(self, image: np.ndarray) -> np.ndarray:
    """
    Fallback: Generate a synthetic depth map based on brightness/shadows.
    This is used when MiDaS is not available.
    """
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray_f = gray.astype(np.float32) / 255.0

    # Darker areas = deeper (shadows in potholes)
    # Invert: brighter = closer, darker = farther
    depth_map = 1.0 - gray_f

    # Apply Gaussian blur to smooth
    depth_map = cv2.GaussianBlur(depth_map, (15, 15), 0)

    # Normalize to [0, 1]
    depth_min = depth_map.min()
    depth_max = depth_map.max()
    if depth_max > depth_min:
        depth_map = (depth_map - depth_min) / (depth_max - depth_min)

    return depth_map.astype(np.float32)

def get_pothole_depth(self, depth_map: np.ndarray,
                      bbox: Tuple[int, int, int, int],
                      reference_depth: float = None) -> Tuple[float, str]:
    """
    Estimate pothole depth from the depth map within the bounding box.

    Args:
        depth_map: Full image depth map
        bbox: (x1, y1, x2, y2) bounding box
        reference_depth: Depth of surrounding road (meters)

    Returns:
        Tuple of (depth_in_cm, depth_category)
    """
    x1, y1, x2, y2 = bbox
    region = depth_map[y1:y2, x1:x2]

    if region.size == 0:
        return 0.0, ""

    # Pothole depth = surrounding road depth - pothole depth (inverted)
    # Since MiDaS gives relative depth, deeper = higher value
    # In road images, potholes appear darker/deeper

    mean_depth = float(np.mean(region))
    min_depth = float(np.min(region))

    # Estimate depth difference from surrounding road
    # Use a ring around the bounding box as reference
    h, w = depth_map.shape
    pad = 20
    x1_ref = max(0, x1 - pad)
    y1_ref = max(0, y1 - pad)
    x2_ref = min(w, x2 + pad)
    y2_ref = min(h, y2 + pad)

    # Extract reference region (excluding pothole area)
    ref_region = depth_map[y1_ref:y2_ref, x1_ref:x2_ref].copy()
    ref_h, ref_w = ref_region.shape

    # Mask out the pothole area from reference
    pothole_h = y2 - y1
    pothole_w = x2 - x1
    py_offset = y1 - y1_ref
    px_offset = x1 - x1_ref

    if px_offset >= 0 and py_offset >= 0 and \
        px_offset + pothole_w <= ref_w and py_offset + pothole_h <= ref_h:
        ref_region[py_offset:py_offset + pothole_h,
                  px_offset:px_offset + pothole_w] = 0

    ref_region = ref_region[ref_region > 0]
    ref_mean = float(np.mean(ref_region)) if len(ref_region) > 0 else mean_depth

    # Depth difference (pothole is deeper = lower relative depth value)
    depth_diff = abs(ref_mean - mean_depth)
    depth_diff = float(min(max(depth_diff, 0.0), 1.0))

```

```

# Map the normalized difference to a plausible physical pothole depth.
# Relative / synthetic depth maps are dimensionless, so this is a
# calibrated estimate: full contrast (1.0) -> 25 cm, which still lands
# in the Deep category but stays within realistic pothole geometry.
# NOTE: For survey-grade accuracy, calibrate this against measured
# ground-truth depths or use stereo/LiDAR (see docs).
max_physical_cm = getattr(config, "DEPTH_MAX_CM_ESTIMATE", 25.0)
depth_cm = depth_diff * max_physical_cm

# Geometric plausibility bound: a routine pothole is wider than it is
# deep. Cap the estimate at ~50% of the pothole's minor real-world
# dimension (derived from the bbox using the same calibrated baseline
# as size estimation). Requires config.PIXEL_TO_CM_BASELINE.
try:
    minor_px = max(1, min(abs(x2 - x1), abs(y2 - y1)))
    map_w = depth_map.shape[1] if depth_map.ndim == 2 else 1
    dens = (getattr(config, "PIXEL_TO_CM_BASELINE", 0.35) *
            (640.0 / map_w if map_w else 1.0))
    minor_cm = minor_px * dens
    depth_cm = min(depth_cm, round(0.5 * minor_cm, 2))
except Exception:
    pass
depth_cm = round(max(depth_cm, 0.0), 2)

# Classify depth
depth_category = self._classify_depth(depth_cm)

return depth_cm, depth_category

def _classify_depth(self, depth_cm: float) -> str:
    """Classify depth into categories."""
    if depth_cm < config.DEPTH_SHALLOW_MAX_CM:
        return "Shallow"
    elif depth_cm < config.DEPTH_MODERATE_MAX_CM:
        return "Moderate"
    else:
        return "Deep"

def create_depth_visualization(self, depth_map: np.ndarray) -> np.ndarray:
    """Create a color-mapped visualization of the depth map."""
    depth_normalized = depth_map.copy()
    depth_min = depth_normalized.min()
    depth_max = depth_normalized.max()
    if depth_max > depth_min:
        depth_normalized = (depth_normalized - depth_min) / (depth_max - depth_min)
    depth_uint8 = (depth_normalized * 255).astype(np.uint8)
    depth_colored = cv2.applyColorMap(depth_uint8, cv2.COLORMAP_JET)
    return depth_colored

def estimate_depth_for_pothole(self, image, bbox):
    """Full pipeline: estimate depth map and extract pothole depth."""
    depth_map = self.estimate_depth(image)
    depth_cm, category = self.get_pothole_depth(depth_map, bbox)
    return DepthResult(
        depth_map=depth_map,
        relative_depth=(depth_map - depth_map.min()) /
            (depth_map.max() - depth_map.min() + 1e-8),
        pothole_depth_cm=depth_cm,
        depth_category=category,
    )

```

=====

src/size_estimator.py | Pixel -> cm size estimation (camera model) + Small/Medium/Large

=====

"""

Size Estimation Module

=====

Estimates pothole size (length, width, area) in real-world units
using pixel measurements and camera calibration parameters.

Classes:

- SizeEstimator: Estimates real-world pothole dimensions

Usage:

```
from src.size_estimator import SizeEstimator
```

```
estimator = SizeEstimator()
```

```
width_cm, height_cm, area_cm2 = estimator.estimate_size(bbox, image_width)
```

"""

```
import numpy as np
```

```
from typing import Tuple, Optional
```

```
from dataclasses import dataclass
```

```
import config
```

```
from .utils import setup_logger
```

```
logger = setup_logger(__name__)
```

```
@dataclass
```

```
class SizeResult:
```

```
    """Container for size estimation results."""
```

```
    width_px: int = 0
```

```
    height_px: int = 0
```

```
    width_cm: float = 0.0
```

```
    height_cm: float = 0.0
```

```
    area_cm2: float = 0.0
```

```
    size_category: str = ""
```

```
    pixel_density_cm_per_px: float = 0.0
```

```
class SizeEstimator:
```

```
    """
```

```
    Real-world size estimation for potholes.
```

```
    Uses camera parameters and reference measurements to convert  
    pixel coordinates to physical dimensions (cm).
```

```
    """
```

```
    def __init__(self,
```

```
        focal_length: float = None,
```

```
        camera_height_cm: float = None,
```

```
        reference_width_cm: float = None):
```

```
        """
```

```
        Initialize size estimator.
```

```
    Args:
```

```
        focal_length: Camera focal length in pixels
```

```
        camera_height_cm: Camera height from road surface (cm)
```

```
        reference_width_cm: Known reference object width (cm)
```

```
        """
```

```
    self.focal_length = focal_length or config.FOCAL_LENGTH_PIXELS
```

```
    self.camera_height_cm = camera_height_cm or config.CAMERA_HEIGHT_CM
```

```
    self.reference_width_cm = reference_width_cm or config.REFERENCE_OBJECT_SIZE_CM
```

```
    def estimate_size(self, bbox: Tuple[int, int, int, int],
```

```
        image_width: int,
```

```
        reference_bbox: Tuple[int, int, int, int] = None) -> SizeResult:
```

```
        """
```

```
        Estimate real-world pothole size from bounding box.
```

```
    Args:
```

```
        bbox: (x1, y1, x2, y2) bounding box in pixels
```

```
        image_width: Width of the source image in pixels
```

```
        reference_bbox: Optional reference object bbox for calibration
```

```
    Returns:
```

```
        SizeResult with all size measurements
```

```
        """
```

```
        x1, y1, x2, y2 = bbox
```

```
        width_px = x2 - x1
```

```
        height_px = y2 - y1
```

```

area_px = width_px * height_px

# Calculate pixel-to-cm conversion factor
pixel_density = self._calculate_pixel_density(
    image_width, reference_bbox
)

width_cm = width_px * pixel_density
height_cm = height_px * pixel_density
area_cm2 = width_cm * height_cm

size_category = self._classify_size(area_cm2)

logger.info(
    f"Size: {width_cm:.1f}x{height_cm:.1f}cm, "
    f"Area: {area_cm2:.1f}cm2, Category: {size_category}"
)

return SizeResult(
    width_px=width_px,
    height_px=height_px,
    width_cm=round(width_cm, 2),
    height_cm=round(height_cm, 2),
    area_cm2=round(area_cm2, 2),
    size_category=size_category,
    pixel_density_cm_per_px=round(pixel_density, 6),
)

def _calculate_pixel_density(self, image_width: int,
                            reference_bbox: Tuple[int, int, int, int] = None
                            ) -> float:
    """
    Calculate pixel-to-cm density.

    Uses reference object if provided, otherwise estimates
    based on camera parameters and geometry.
    """
    if reference_bbox is not None:
        ref_x1, ref_y1, ref_x2, ref_y2 = reference_bbox
        ref_width_px = abs(ref_x2 - ref_x1)
        if ref_width_px > 0:
            return self.reference_width_cm / ref_width_px

    # Fallback: calibrated baseline estimate.
    # IMPORTANT: For accurate measurements, pass a reference_bbox with a
    # known object width, or calibrate config.PIXEL_TO_CM_BASELINE for
    # your specific camera and mounting height.
    baseline_640 = config.PIXEL_TO_CM_BASELINE
    # Same scene at higher resolution => each pixel covers LESS ground.
    # scale_factor = W / 640, so cm-per-pixel = baseline / scale_factor.
    scale_factor = image_width / 640.0 if image_width > 0 else 1.0
    pixel_to_cm = baseline_640 / scale_factor

    return pixel_to_cm

def _classify_size(self, area_cm2: float) -> str:
    """Classify pothole size category based on area."""
    if area_cm2 < config.SIZE_SMALL_MAX_CM2:
        return "Small"
    elif area_cm2 < config.SIZE_MEDIUM_MAX_CM2:
        return "Medium"
    else:
        return "Large"

def estimate_depth_from_size_ratio(self, bbox: Tuple[int, int, int, int],
                                   depth_value: float) -> float:
    """
    Rough depth estimate based on depth map value and size ratio.

    Args:
        bbox: Bounding box
        depth_value: Depth map value at pothole center

    Returns:
        Estimated depth in cm
    """
    # This is a supplementary method for cross-validation
    x1, y1, x2, y2 = bbox
    width_px = x2 - x1
    height_px = y2 - y1

    # Larger potholes tend to be deeper (empirical observation)

```

```
area_factor = min(width_px * height_px / 10000.0, 5.0)
depth_estimate = depth_value * 5.0 * area_factor # Scale to cm

return round(depth_estimate, 2)

def batch_estimate(self, bboxes: list, image_width: int) -> list:
    """Estimate sizes for a batch of bounding boxes."""
    results = []
    for bbox in bboxes:
        result = self.estimate_size(bbox, image_width)
        results.append(result)
    return results
```

src/severity.py | Severity scoring (0-100) + matrix classification + priority levels

```
"""
Severity Classification Module
=====
Combines size and depth measurements to classify pothole
severity and assign maintenance priority levels.

Classes:
- SeverityClassifier: Classifies pothole severity
- SeverityResult: Container for classification output

Usage:
    from src.severity import SeverityClassifier

    classifier = SeverityClassifier()
    result = classifier.classify(size_result, depth_result)
    print(result.severity, result.priority)
"""

from typing import Tuple, Optional
from dataclasses import dataclass

import config
from .utils import setup_logger

logger = setup_logger(__name__)

@dataclass
class SeverityResult:
    """Container for severity classification results."""
    severity: str # Critical, High, Medium, Low
    size_category: str # Small, Medium, Large
    depth_category: str # Shallow, Moderate, Deep
    priority: int # 1=Critical, 4=Low (lower = more urgent)
    color_bgr: Tuple[int, int, int] # BGR color for visualization
    response_time_hours: int # Recommended response deadline
    recommendation: str # Maintenance recommendation text
    score: float = 0.0 # Numerical severity score (0-100)

class SeverityClassifier:
    """
    Classify pothole severity based on size and depth.

    Uses a rule-based system combined with configurable thresholds
    from the config module.
    """

    def __init__(self):
        """Initialize the severity classifier with config thresholds."""
        self.size_thresholds = config.SIZE_CATEGORIES
        self.depth_thresholds = config.DEPTH_CATEGORIES
        self.severity_levels = config.SEVERITY_LEVELS
        logger.info("SeverityClassifier initialized")

    def classify(self, size_result=None, depth_cm: float = None,
                depth_category: str = None, area_cm2: float = None,
                size_category: str = None) -> SeverityResult:
        """
        Classify pothole severity.

        Can accept either SizeResult/DepthResult objects or raw values.

        Args:
            size_result: SizeResult object (optional)
            depth_cm: Depth in centimeters
            depth_category: Depth category string
            area_cm2: Pothole area in cm2
            size_category: Size category string

        Returns:
            SeverityResult with full classification
        """
        # Extract size info
        if size_result is not None:
            area_cm2 = size_result.area_cm2
            size_category = size_result.size_category
        elif area_cm2 is None or size_category is None:
            area_cm2 = 0
```

```

        size_category = "Unknown"

# Extract depth info
if depth_cm is not None and depth_category is None:
    depth_category = self._category_from_depth(depth_cm)

if depth_cm is None:
    depth_cm = 0
if depth_category is None:
    depth_category = "Shallow"

# Calculate severity score (0-100)
score = self._calculate_score(area_cm2, depth_cm, size_category, depth_category)

# Determine severity level
severity, priority, color, response_hours = self._determine_severity(score)

# Generate recommendation
recommendation = self._generate_recommendation(
    severity, size_category, depth_category
)

logger.info(
    f"Severity: {severity} (Priority {priority}), "
    f"Score: {score:.1f}, Size: {size_category}, Depth: {depth_category}"
)

return SeverityResult(
    severity=severity,
    size_category=size_category,
    depth_category=depth_category,
    priority=priority,
    color_bgr=color,
    response_time_hours=response_hours,
    recommendation=recommendation,
    score=round(score, 2),
)

def _calculate_score(self, area_cm2: float, depth_cm: float,
                    size_category: str, depth_category: str) -> float:
    """
    Calculate severity score (0-100).

    Combines size and depth with weighted scoring.
    """
    # Size score (0-50)
    if size_category == "Small":
        size_score = min(area_cm2 / config.SIZE_SMALL_MAX_CM2 * 15, 15)
    elif size_category == "Medium":
        ratio = (area_cm2 - config.SIZE_SMALL_MAX_CM2) / \
            (config.SIZE_MEDIUM_MAX_CM2 - config.SIZE_SMALL_MAX_CM2)
        size_score = 15 + ratio * 25 # 15-40
    else:
        ratio = min((area_cm2 - config.SIZE_MEDIUM_MAX_CM2) / 2000, 1.0)
        size_score = 40 + ratio * 10 # 40-50

    # Depth score (0-50)
    if depth_category == "Shallow":
        depth_score = min(depth_cm / config.DEPTH_SHALLOW_MAX_CM * 10, 10)
    elif depth_category == "Moderate":
        ratio = (depth_cm - config.DEPTH_SHALLOW_MAX_CM) / \
            (config.DEPTH_MODERATE_MAX_CM - config.DEPTH_SHALLOW_MAX_CM)
        depth_score = 10 + ratio * 30 # 10-40
    else:
        ratio = min((depth_cm - config.DEPTH_MODERATE_MAX_CM) / 5.0, 1.0)
        depth_score = 40 + ratio * 10 # 40-50

    return size_score + depth_score

def _determine_severity(self, score: float) -> Tuple[str, int, Tuple[int, int, int], int]:
    """Map severity score to severity level, priority, and color."""
    if score >= 80:
        return ("Critical", 1, (0, 0, 255), 24)
    elif score >= 60:
        return ("High", 2, (0, 165, 255), 168)
    elif score >= 35:
        return ("Medium", 3, (0, 255, 255), 720)
    else:
        return ("Low", 4, (0, 255, 0), 2160)

def _category_from_depth(self, depth_cm: float) -> str:
    """Determine depth category from raw depth value."""

```

```

    if depth_cm < config.DEPTH_SHALLOW_MAX_CM:
        return "Shallow"
    elif depth_cm < config.DEPTH_MODERATE_MAX_CM:
        return "Moderate"
    else:
        return "Deep"

def _generate_recommendation(self, severity: str,
                             size_category: str,
                             depth_category: str) -> str:
    """Generate a maintenance recommendation string."""
    if severity == "Critical":
        return (f"CRITICAL: Immediate repair required. "
                f"Large ({size_category}) and/or deep ({depth_category}) pothole "
                f"posing severe safety risk. Dispatch crew within 24h.")
    elif severity == "High":
        return (f"HIGH PRIORITY: Schedule repair within 1 week. "
                f"Size: {size_category}, Depth: {depth_category}. "
                f"Monitor for rapid deterioration.")
    elif severity == "Medium":
        return (f"MEDIUM PRIORITY: Plan repair within 1 month. "
                f"Size: {size_category}, Depth: {depth_category}. "
                f"Include in quarterly maintenance schedule.")
    else:
        return (f"LOW PRIORITY: Routine monitoring. "
                f"Size: {size_category}, Depth: {depth_category}. "
                f"Include in annual maintenance planning.")

def classify_matrix(self, size_category: str, depth_category: str) -> SeverityResult:
    """
    Classify using a pre-defined severity matrix.

    This provides an alternative rule-based approach
    for quick severity assignment.
    """
    matrix = {
        ("Small", "Shallow"): ("Low", 4, 2160),
        ("Small", "Moderate"): ("Medium", 3, 720),
        ("Small", "Deep"): ("High", 2, 168),
        ("Medium", "Shallow"): ("Medium", 3, 720),
        ("Medium", "Moderate"): ("High", 2, 168),
        ("Medium", "Deep"): ("Critical", 1, 24),
        ("Large", "Shallow"): ("High", 2, 168),
        ("Large", "Moderate"): ("Critical", 1, 24),
        ("Large", "Deep"): ("Critical", 1, 24),
    }

    if (size_category, depth_category) in matrix:
        sev, pri, hours = matrix[(size_category, depth_category)]
    else:
        sev, pri, hours = ("Low", 4, 2160)

    colors = {
        "Critical": (0, 0, 255),
        "High": (0, 165, 255),
        "Medium": (0, 255, 255),
        "Low": (0, 255, 0),
    }

    recommendation = self._generate_recommendation(sev, size_category, depth_category)

    return SeverityResult(
        severity=sev,
        size_category=size_category,
        depth_category=depth_category,
        priority=pri,
        color_bgr=colors[sev],
        response_time_hours=hours,
        recommendation=recommendation,
        score=0.0,
    )

```

```
=====
src/utils.py | Helpers: logging, image IO, drawing boxes, IoU, report generation
=====
```

```
"""
```

```
Utility functions for the Road Pothole Detection System.
```

```
Provides helper functions for:
```

- File I/O and path management
- Image conversion and visualization
- Coordinate transformations
- Logging setup
- Data serialization

```
"""
```

```
import os
import cv2
import json
import logging
import numpy as np
from datetime import datetime
from typing import Tuple, List, Optional, Dict, Any
```

```
import config
```

```
# =====
```

```
# Logging Setup
```

```
# =====
```

```
def setup_logger(name: str, log_file: Optional[str] = None, level: int = logging.INFO) -> logging.Logger:
```

```
    """
```

```
    Set up a logger with console and file handlers.
```

```
    Args:
```

```
        name: Logger name
        log_file: Optional path to log file
        level: Logging level
```

```
    Returns:
```

```
        Configured logger instance
```

```
    """
```

```
    logger = logging.getLogger(name)
    logger.setLevel(level)
```

```
    # Avoid duplicate handlers
```

```
    if logger.handlers:
        return logger
```

```
    formatter = logging.Formatter(
        "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S"
    )
```

```
    # Console handler
```

```
    console_handler = logging.StreamHandler()
    console_handler.setLevel(level)
    console_handler.setFormatter(formatter)
    logger.addHandler(console_handler)
```

```
    # File handler
```

```
    if log_file:
        file_handler = logging.FileHandler(log_file)
        file_handler.setLevel(level)
        file_handler.setFormatter(formatter)
        logger.addHandler(file_handler)
```

```
    return logger
```

```
# =====
```

```
# Image Utilities
```

```
# =====
```

```
def load_image(image_path: str) -> Optional[np.ndarray]:
```

```
    """
```

```
    Load an image from file path.
```

```
    Args:
```

```
        image_path: Path to image file
```

```
    Returns:
```

```
        Image as numpy array (BGR format) or None if loading fails
```

```
    """
```

```
    if not os.path.exists(image_path):
```

```

        raise FileNotFoundError(f"Image not found: {image_path}")

    image = cv2.imread(image_path)
    if image is None:
        raise ValueError(f"Failed to load image: {image_path}")

    return image

def load_image_from_bytes(image_bytes: bytes) -> Optional[np.ndarray]:
    """
    Load an image from raw bytes.

    Args:
        image_bytes: Raw image bytes

    Returns:
        Image as numpy array (BGR format) or None
    """
    nparr = np.frombuffer(image_bytes, np.uint8)
    image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    return image

def save_image(image: np.ndarray, output_path: str) -> bool:
    """
    Save an image to file.

    Args:
        image: Image as numpy array
        output_path: Output file path

    Returns:
        True if saved successfully, False otherwise
    """
    dir_path = os.path.dirname(output_path)
    if dir_path:
        os.makedirs(dir_path, exist_ok=True)
    return cv2.imwrite(output_path, image)

def image_to_base64(image: np.ndarray) -> str:
    """Convert a numpy image array to base64 string."""
    import base64
    _, buffer = cv2.imencode(".jpg", image)
    return base64.b64encode(buffer).decode("utf-8")

def base64_to_image(base64_str: str) -> np.ndarray:
    """Convert a base64 string to numpy image array."""
    import base64
    image_data = base64.b64decode(base64_str)
    nparr = np.frombuffer(image_data, np.uint8)
    return cv2.imdecode(nparr, cv2.IMREAD_COLOR)

def resize_image(image: np.ndarray, max_dim: int = 640) -> np.ndarray:
    """
    Resize image while maintaining aspect ratio.

    Args:
        image: Input image
        max_dim: Maximum dimension (height or width)

    Returns:
        Resized image
    """
    h, w = image.shape[:2]
    scale = min(max_dim / w, max_dim / h, 1.0)
    new_w, new_h = int(w * scale), int(h * scale)
    return cv2.resize(image, (new_w, new_h), interpolation=cv2.INTER_AREA)

# =====
# Coordinate & Bounding Box Utilities
# =====
def extract_bounding_boxes(results, image_shape: Tuple[int, int]) -> List[Dict[str, Any]]:
    """
    Extract bounding boxes from YOLO detection results.

    Args:
        results: YOLO detection results
    """

```

```

        image_shape: (height, width) of the source image

Returns:
    List of bounding box dictionaries
    """
    boxes = []
    img_h, img_w = image_shape[:2]

    if results.bboxes is not None:
        for box in results.bboxes:
            xyxy = box.xyxy[0].cpu().numpy()
            conf = float(box.conf.cpu().numpy()[0])
            cls_id = int(box.cls.cpu().numpy()[0])

            x1, y1, x2, y2 = xyxy
            width_px = int(x2 - x1)
            height_px = int(y2 - y1)

            x1 = max(0, int(x1))
            y1 = max(0, int(y1))
            x2 = min(img_w - 1, int(x2))
            y2 = min(img_h - 1, int(y2))

            class_name = config.CLASS_NAMES.get(cls_id, f"class_{cls_id}")

            boxes.append({
                "bbox": [x1, y1, x2, y2],
                "confidence": conf,
                "class_id": cls_id,
                "class_name": class_name,
                "width_px": width_px,
                "height_px": height_px,
                "center": [(x1 + x2) // 2, (y1 + y2) // 2],
            })

    return boxes

def pixels_to_cm(pixel_width: float, pixel_height: float,
                 reference_width_px: float = None,
                 reference_width_cm: float = None,
                 distance_m: float = 2.0) -> Tuple[float, float]:
    """Convert pixel measurements to real-world centimeters."""
    if reference_width_px and reference_width_cm and reference_width_px > 0:
        pixel_to_cm = reference_width_cm / reference_width_px
        width_cm = pixel_width * pixel_to_cm
        height_cm = pixel_height * pixel_to_cm
    else:
        f = config.FOCAL_LENGTH_PIXELS
        pixel_to_cm = (distance_m * 100) / f
        width_cm = pixel_width * pixel_to_cm
        height_cm = pixel_height * pixel_to_cm
    return width_cm, height_cm

def draw_bounding_boxes(image: np.ndarray, detections: List[Dict[str, Any]],
                        thickness: int = 2) -> np.ndarray:
    """Draw bounding boxes with labels on image."""
    img = image.copy()

    for det in detections:
        x1, y1, x2, y2 = det["bbox"]
        conf = det["confidence"]
        label = det.get("class_name", "Object")
        severity = det.get("severity", "")
        size_cat = det.get("size_category", "")

        if severity == "Critical":
            color = (0, 0, 255)
        elif severity == "High":
            color = (0, 165, 255)
        elif severity == "Medium":
            color = (0, 255, 255)
        else:
            color = (0, 255, 0)

        cv2.rectangle(img, (x1, y1), (x2, y2), color, thickness)

        label_text = f"{label}: {conf:.2f}"
        if size_cat:
            label_text += f" [{size_cat}]"

```

```

(w, h), _ = cv2.getTextSize(label_text, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 1)
cv2.rectangle(img, (x1, y1 - 20), (x1 + w, y1), color, -1)
cv2.putText(img, label_text, (x1, y1 - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

if severity:
    sev_text = f"Severity: {severity}"
    cv2.rectangle(img, (x1, y2), (x1 + 120, y2 + 20), color, -1)
    cv2.putText(img, sev_text, (x1 + 5, y2 + 15),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

return img

# =====
# Report Generation
# =====
def generate_report_json(detections: List[Dict[str, Any]],
                        image_path: str = "",
                        gps_coords: Tuple[float, float] = None) -> Dict[str, Any]:
    """Generate a JSON-serializable report from detections."""
    report = {
        "timestamp": datetime.now().isoformat(),
        "image_path": image_path,
        "gps_coordinates": {"lat": gps_coords[0], "lng": gps_coords[1]} if gps_coords else None,
        "total_potholes_detected": len(detections),
        "potholes": detections,
        "summary": {
            "critical_count": sum(1 for d in detections if d.get("severity") == "Critical"),
            "high_count": sum(1 for d in detections if d.get("severity") == "High"),
            "medium_count": sum(1 for d in detections if d.get("severity") == "Medium"),
            "low_count": sum(1 for d in detections if d.get("severity") == "Low"),
        }
    }
    return report

def format_severity_report(report: Dict[str, Any]) -> str:
    """Format a report dictionary into a human-readable string."""
    lines = [
        "-" * 60,
        "POTHOLE DETECTION & SEVERITY REPORT",
        "-" * 60,
        f"Timestamp: {report['timestamp']}",
        f"Image: {report.get('image_path', 'N/A')}",
        f"GPS: {report.get('gps_coordinates', 'N/A')}",
        f"Total Potholes Detected: {report['total_potholes_detected']}",
        "",
        "SEVERITY SUMMARY:",
        f" [CRITICAL]: {report['summary']['critical_count']}",
        f" [HIGH]:      {report['summary']['high_count']}",
        f" [MEDIUM]:    {report['summary']['medium_count']}",
        f" [LOW]:       {report['summary']['low_count']}",
        "",
        "DETAILED FINDINGS:",
        "-" * 60,
    ]

    for i, p in enumerate(report["potholes"], 1):
        lines.append(f"\nPothole #{i}:")
        lines.append(f"  Location (px): bbox={p['bbox']}")
        lines.append(f"  Confidence: {p['confidence']:.4f}")
        lines.append(f"  Size: {p.get('size_cm2', 'N/A')} cm2 ({p.get('size_category', 'N/A')})")
        lines.append(f"  Dimensions: {p.get('width_cm', 'N/A')}cm x {p.get('height_cm', 'N/A')}cm")
        lines.append(f"  Depth Est.: {p.get('depth_cm', 'N/A')} cm ({p.get('depth_category', 'N/A')})")
        lines.append(f"  Severity: {p.get('severity', 'Unknown')}")
        lines.append(f"  Priority: {p.get('priority', 'N/A')}")

    lines.append("\n" + "-" * 60)
    return "\n".join(lines)

def save_json(data: Dict[str, Any], filepath: str) -> None:
    """Save dictionary as JSON file."""
    dir_path = os.path.dirname(filepath)
    if dir_path:
        os.makedirs(dir_path, exist_ok=True)
    with open(filepath, "w") as f:
        json.dump(data, f, indent=2, default=str)

def load_json(filepath: str) -> Dict[str, Any]:

```

```

        """Load JSON file as dictionary."""
        if not os.path.exists(filepath):
            return {}
        with open(filepath, "r") as f:
            return json.load(f)

# =====
# Performance Metrics
# =====
def calculate_iou(box1: List[int], box2: List[int]) -> float:
    """Calculate Intersection over Union (IoU) between two bounding boxes."""
    x1_i = max(box1[0], box2[0])
    y1_i = max(box1[1], box2[1])
    x2_i = min(box1[2], box2[2])
    y2_i = min(box1[3], box2[3])

    intersection_area = max(0, x2_i - x1_i) * max(0, y2_i - y1_i)

    box1_area = (box1[2] - box1[0]) * (box1[3] - box1[1])
    box2_area = (box2[2] - box2[0]) * (box2[3] - box2[1])

    union_area = box1_area + box2_area - intersection_area

    if union_area == 0:
        return 0.0

    return intersection_area / union_area

# =====
# Validation Utilities
# =====
def is_valid_image(filepath: str) -> bool:
    """Check if file is a valid image."""
    try:
        img = load_image(filepath)
        return img is not None
    except Exception:
        return False

def get_file_size_mb(filepath: str) -> float:
    """Get file size in megabytes."""
    if os.path.exists(filepath):
        return os.path.getsize(filepath) / (1024 * 1024)
    return 0.0

def allowed_file(filename: str) -> bool:
    """Check if file extension is allowed."""
    return "." in filename and \
        filename.rsplit(".", 1)[1].lower() in config.ALLOWED_EXTENSIONS

```

```
=====
api/app.py | Flask REST API: /api/analyze, /api/detect, /api/reports, /dashboard
=====
```

```
"""
```

```
Flask API Application for Pothole Detection
```

```
=====
```

```
REST API endpoints for automated pothole detection and severity analysis.
```

```
Endpoints:
```

```
GET / - API info
GET /dashboard - Web dashboard
POST /api/detect - Detect potholes in uploaded image
POST /api/analyze - Full analysis (detection + size + depth + severity)
GET /api/health - Health check
GET /api/reports - List reports
GET /api/report/<id> - Get specific report
```

```
Usage:
```

```
python api/app.py
or
python main.py --web
```

```
"""
```

```
import os
import io
import json
import sys
import uuid
```

```
# Ensure the project root is on sys.path so that `python api/app.py` works.
# Running a script inside a subpackage only puts that subpackage's directory
# on sys.path by default, which would break `import config` and `import src`.
PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)
```

```
import cv2
import numpy as np
from datetime import datetime
from typing import Tuple, Optional
from glob import glob
from threading import Thread
```

```
from flask import Flask, request, jsonify, render_template, \
    send_from_directory, Response, url_for
from flask_cors import CORS
from werkzeug.utils import secure_filename
```

```
import config
from src.detector import PotholeDetector, DetectionResult
from src.depth_estimator import DepthEstimator
from src.size_estimator import SizeEstimator
from src.severity import SeverityClassifier
from main import PotholeAnalysisPipeline
from src.utils import setup_logger, load_image_from_bytes, save_image, \
    image_to_base64, save_json, allowed_file
```

```
logger = setup_logger("api", config.LOG_FILE)
```

```
app = Flask(
    __name__,
    template_folder=os.path.join(os.path.dirname(__file__), "..", "frontend", "templates"),
    static_folder=os.path.join(os.path.dirname(__file__), "..", "frontend", "static"),
    static_url_path="/static",
)
app.config["MAX_CONTENT_LENGTH"] = config.MAX_CONTENT_LENGTH_MB * 1024 * 1024
app.config["SECRET_KEY"] = config.SECRET_KEY
app.config["UPLOAD_FOLDER"] = config.UPLOADS_DIR
CORS(app)
```

```
# Ensure upload directory exists
os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)
```

```
# Lazy-load pipeline (loaded on first request to handle model downloads)
_pipeline = None
```

```
def get_pipeline() -> PotholeAnalysisPipeline:
    """Lazy-initialize the analysis pipeline."""
    global _pipeline
    if _pipeline is None:
        logger.info("Initializing analysis pipeline...")
```

```

        _pipeline = PotholeAnalysisPipeline()
        logger.info("Pipeline ready")
    return _pipeline

# =====
# Routes
# =====
@app.route("/")
def index():
    """API root endpoint."""
    return jsonify({
        "service": "Pothole Detection & Severity Analysis API",
        "version": "1.0.0",
        "endpoints": {
            "/api/health": "GET - Health check",
            "/api/detect": "POST - Upload image, get detection results",
            "/api/analyze": "POST - Full analysis (detect + size + depth + severity)",
            "/dashboard": "GET - Web dashboard",
            "/api/reports": "GET - List all reports",
        },
    })

@app.route("/dashboard")
def dashboard():
    """Serve the web dashboard."""
    return render_template("index.html")

@app.route("/api/health")
def health_check():
    """Health check endpoint."""
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.now().isoformat(),
        "device": config.DEFAULT_DEVICE,
    })

@app.route("/api/detect", methods=["POST"])
def detect_potholes():
    """
    Detect potholes in an uploaded image.

    Expects: multipart/form-data with 'image' file
    Optional: 'gps_lat', 'gps_lng' as form fields

    Returns:
        JSON with detection results and annotated image (base64)
    """
    if "image" not in request.files:
        return jsonify({"error": "No image file provided"}), 400

    file = request.files["image"]
    if file.filename == "":
        return jsonify({"error": "No file selected"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "File type not allowed"}), 400

    # Load image
    image_bytes = file.read()
    image = load_image_from_bytes(image_bytes)

    if image is None:
        return jsonify({"error": "Failed to load image"}), 400

    # Get GPS coords if provided
    gps_lat = request.form.get("gps_lat", type=float)
    gps_lng = request.form.get("gps_lng", type=float)
    gps = (gps_lat, gps_lng) if gps_lat and gps_lng else None

    try:
        pipeline = get_pipeline()
        result = pipeline.analyze_image(image, gps_coords=gps, visualize=True)

        # Save annotated image
        filename = secure_filename(file.filename)
        unique_id = str(uuid.uuid4())[:8]
        annotated_filename = f"{os.path.splitext(filename)[0]}_{unique_id}_annotated.jpg"
        annotated_path = os.path.join(app.config["UPLOAD_FOLDER"], annotated_filename)

```

```

        save_image(result["annotated_image"], annotated_path)

    # Save report
    report_filename = f"{os.path.splitext(filename)[0]}_{unique_id}_report.json"
    report_path = os.path.join(app.config["UPLOAD_FOLDER"], report_filename)
    save_json(result["report"], report_path)

    # Add paths to response
    result["report"]["annotated_image_path"] = annotated_path
    result["report"]["report_path"] = report_path
    result["report"]["report_id"] = unique_id

    return jsonify(result["report"])

except Exception as e:
    logger.error(f"Detection failed: {e}")
    return jsonify({"error": str(e)}), 500

@app.route("/api/analyze", methods=["POST"])
def analyze_pothole():
    """
    Full analysis: detection + size + depth + severity.

    Expects: multipart/form-data with 'image' file
    Optional: 'gps_lat', 'gps_lng' as form fields

    Returns:
        JSON with complete analysis report and base64 annotated image
    """
    if "image" not in request.files:
        return jsonify({"error": "No image file provided"}), 400

    file = request.files["image"]
    if file.filename == "":
        return jsonify({"error": "No file selected"}), 400

    image_bytes = file.read()
    image = load_image_from_bytes(image_bytes)

    if image is None:
        return jsonify({"error": "Failed to load image"}), 400

    gps_lat = request.form.get("gps_lat", type=float)
    gps_lng = request.form.get("gps_lng", type=float)
    gps = (gps_lat, gps_lng) if gps_lat and gps_lng else None

    try:
        pipeline = get_pipeline()
        result = pipeline.analyze_image(image, gps_coords=gps, visualize=True)

        # Convert annotated image to base64
        annotated_b64 = image_to_base64(result["annotated_image"])

        response = {
            "status": "success",
            "timestamp": datetime.now().isoformat(),
            "image_info": {
                "height": image.shape[0],
                "width": image.shape[1],
            },
            "detections": result["detections"],
            "annotated_image_base64": annotated_b64,
            "report": result["report"],
            "processing_time_seconds": result["processing_time_seconds"],
        }

        return jsonify(response)

    except Exception as e:
        logger.error(f"Analysis failed: {e}")
        return jsonify({"error": str(e), "status": "error"}), 500

@app.route("/api/reports", methods=["GET"])
def list_reports():
    """List all saved reports."""
    upload_dir = app.config["UPLOAD_FOLDER"]
    reports = []

    for f in glob(os.path.join(upload_dir, "*_report.json")):
        try:

```

```

        with open(f, "r") as fh:
            report = json.load(fh)
            report["report_file"] = os.path.basename(f)
            report["annotated_image_file"] = os.path.basename(
                f).replace("_report.json", "_annotated.jpg")
            reports.append(report)
    except Exception as e:
        logger.error(f"Failed to read report {f}: {e}")

    return jsonify({"reports": reports, "count": len(reports)})

@app.route("/api/report/<report_id>", methods=["GET"])
def get_report(report_id):
    """Get a specific report by ID."""
    upload_dir = app.config["UPLOAD_FOLDER"]

    for f in glob(os.path.join(upload_dir, f"*_{report_id}_report.json")):
        with open(f, "r") as fh:
            report = json.load(fh)
            return jsonify(report)

    return jsonify({"error": "Report not found"}), 404

@app.route("/api/visualize", methods=["POST"])
def visualize_detections_endpoint():
    """Endpoint to just get the annotated image without full analysis."""
    if "image" not in request.files:
        return jsonify({"error": "No image file provided"}), 400

    file = request.files["image"]
    image_bytes = file.read()
    image = load_image_from_bytes(image_bytes)

    if image is None:
        return jsonify({"error": "Failed to load image"}), 400

    try:
        pipeline = get_pipeline()
        result = pipeline.analyze_image(image, gps_coords=None, visualize=True)
        annotated_b64 = image_to_base64(result["annotated_image"])

        return jsonify({
            "status": "success",
            "annotated_image_base64": annotated_b64,
            "detections": result["detections"],
        })

    except Exception as e:
        logger.error(f"Visualization failed: {e}")
        return jsonify({"error": str(e)}), 500

@app.errorhandler(413)
def request_entity_too_large(error):
    return jsonify({"error": "File too large (max 50MB)"}), 413

@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "Endpoint not found"}), 404

@app.errorhandler(500)
def internal_error(error):
    return jsonify({"error": "Internal server error"}), 500

if __name__ == "__main__":
    app.run(
        host=config.API_HOST,
        port=config.API_PORT,
        debug=config.API_DEBUG,
    )

```

```
=====
train.py | YOLOv8 training script (dataset config, train, validate, export)
=====
```

```
"""
YOLOv8 Training Script for Pothole Detection
=====
```

```
Trains a custom YOLOv8 model on pothole datasets.
```

```
Usage:
```

```
# 1. Create a dataset config template
python train.py --create-config
```

```
# 2. Prepare dataset in YOLO format:
```

```
#   dataset/
#   images/train/*.jpg
#   images/val/*.jpg
#   labels/train/*.txt
#   labels/val/*.txt
#   dataset.yaml
```

```
# 3. Train:
```

```
python train.py --data dataset.yaml --epochs 100 --model yolov8s.pt
```

```
# 4. Validate:
```

```
python train.py --validate runs/train/pothole_model/weights/best.pt
```

```
# 5. Export to ONNX:
```

```
python train.py --export runs/train/pothole_model/weights/best.pt
```

```
"""
```

```
import os
import sys
import argparse
import shutil
import yaml
```

```
try:
```

```
    from ultralytics import YOLO
    ULTRALYTICS_AVAILABLE = True
```

```
except ImportError:
```

```
    ULTRALYTICS_AVAILABLE = False
```

```
import config
```

```
from src.utils import setup_logger
```

```
logger = setup_logger("train", config.LOG_FILE)
```

```
# Default dataset configuration template
```

```
DATASET_TEMPLATE = {
    "path": "datasets/pothole_data",
    "train": "images/train",
    "val": "images/val",
    "test": "images/test",
    "names": {0: "Pothole"},
}
```

```
def create_dataset_yaml(output_path: str, dataset_root: str = None,
                        class_names: dict = None) -> str:
```

```
    """
```

```
    Create a YOLO dataset configuration YAML file.
```

```
    Args:
```

```
        output_path: Path where to write the YAML
        dataset_root: Root directory of the dataset
        class_names: Dictionary of {class_id: class_name}
```

```
    Returns:
```

```
        Path to created YAML file
```

```
    """
```

```
    cfg = DATASET_TEMPLATE.copy()
```

```
    if dataset_root:
```

```
        cfg["path"] = dataset_root
```

```
    if class_names:
```

```
        cfg["names"] = class_names
```

```
    dir_path = os.path.dirname(output_path)
```

```
    if dir_path:
```

```
        os.makedirs(dir_path, exist_ok=True)
```

```
    with open(output_path, "w") as f:
```

```

        yaml.dump(cfg, f, default_flow_style=False)

    logger.info(f"Dataset config created: {output_path}")
    return output_path

def train_model(data_yaml, model_name="yolov8s.pt", epochs=100,
                batch_size=16, img_size=640, device=None,
                project="runs/train", name="pothole_model",
                resume=False, patience=20):
    """
    Train a YOLOv8 model for pothole detection.

    Args:
        data_yaml: Path to dataset configuration YAML
        model_name: Base model to fine-tune (yolov8n/s/m/l/x.pt)
        epochs: Number of training epochs
        batch_size: Batch size
        img_size: Input image size
        device: Device ('cuda', 'cpu', or None for auto)
        project: Output project directory
        name: Run name
        resume: Resume from last checkpoint
        patience: Early stopping patience

    Returns:
        Training results
    """
    if not ULTRALYTICS_AVAILABLE:
        raise ImportError(
            "Ultralytics is required for training. "
            "Install with: pip install ultralytics"
        )

    if not os.path.exists(data_yaml):
        raise FileNotFoundError(f"Dataset config not found: {data_yaml}")

    if device is None:
        device = config.DEFAULT_DEVICE

    logger.info("=" * 60)
    logger.info("STARTING YOLOv8 TRAINING")
    logger.info("=" * 60)
    logger.info(f"  Base model:    {model_name}")
    logger.info(f"  Dataset:      {data_yaml}")
    logger.info(f"  Epochs:       {epochs}")
    logger.info(f"  Batch size:   {batch_size}")
    logger.info(f"  Image size:   {img_size}")
    logger.info(f"  Device:       {device}")
    logger.info(f"  Output:       {project}/{name}")

    model = YOLO(model_name)

    results = model.train(
        data=data_yaml,
        epochs=epochs,
        batch=batch_size,
        imgsz=img_size,
        device=device,
        project=project,
        name=name,
        resume=resume,
        patience=patience,
        mosaic=1.0,
        mixup=0.1,
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
        degrees=15.0,
        translate=0.1,
        scale=0.5,
        fliplr=0.5,
        optimizer="AdamW",
        lr0=0.001,
        lrf=0.01,
        weight_decay=0.0005,
        warmup_epochs=3.0,
        verbose=True,
    )

    best_model = os.path.join(project, name, "weights", "best.pt")
    if os.path.exists(best_model):

```

```

        dest = config.YOLO_MODEL_PATH
        os.makedirs(os.path.dirname(dest), exist_ok=True)
        shutil.copy2(best_model, dest)
        logger.info(f"Best model copied to: {dest}")

    logger.info("Training complete!")
    return results

def validate_model(model_path, data_yaml, device=None):
    """Validate a trained model on the validation set."""
    if device is None:
        device = config.DEFAULT_DEVICE

    logger.info(f"Validating model: {model_path}")
    model = YOLO(model_path)
    metrics = model.val(data=data_yaml, device=device)

    logger.info(f"    mAP@0.5:      {metrics.box.map50:.4f}")
    logger.info(f"    mAP@0.5:0.95: {metrics.box.map:.4f}")
    logger.info(f"    Precision:     {metrics.box.mp:.4f}")
    logger.info(f"    Recall:        {metrics.box.mr:.4f}")

    return metrics

def export_model(model_path, export_format="onnx"):
    """
    Export a trained model to a deployment format.

    Args:
        model_path: Path to trained .pt model
        export_format: onnx, tflite, torchscript, coreml, etc.
    """
    logger.info(f"Exporting {model_path} to {export_format}...")
    model = YOLO(model_path)
    model.export(format=export_format)
    logger.info(f"Export complete: {export_format}")

def main():
    """Command-line entry point for training."""
    parser = argparse.ArgumentParser(
        description="Train YOLOv8 model for pothole detection"
    )
    parser.add_argument("--data", type=str, default="dataset.yaml",
                        help="Path to dataset YAML config")
    parser.add_argument("--model", type=str, default="yolov8s.pt",
                        help="Base model (yolov8n/s/m/l/x.pt)")
    parser.add_argument("--epochs", type=int, default=100,
                        help="Number of training epochs")
    parser.add_argument("--batch", type=int, default=16,
                        help="Batch size")
    parser.add_argument("--imgsz", type=int, default=640,
                        help="Input image size")
    parser.add_argument("--device", type=str, default=None,
                        help="Device: cuda, cpu, or 0,1 for multi-GPU")
    parser.add_argument("--project", type=str, default="runs/train",
                        help="Output project directory")
    parser.add_argument("--name", type=str, default="pothole_model",
                        help="Run name")
    parser.add_argument("--resume", action="store_true",
                        help="Resume from last checkpoint")
    parser.add_argument("--patience", type=int, default=20,
                        help="Early stopping patience")
    parser.add_argument("--validate", type=str, default=None,
                        help="Validate a model at this path (skip training)")
    parser.add_argument("--export", type=str, default=None,
                        help="Export model to format (onnx, tflite, etc.)")
    parser.add_argument("--create-config", action="store_true",
                        help="Create a template dataset.yaml file")

    args = parser.parse_args()

    if args.create_config:
        path = create_dataset_yaml("dataset.yaml")
        print(f"\nDataset config template created: {path}")
        print("Edit it to point to your dataset, then run training.")
        return

    if args.export:
        export_model(args.export, "onnx")

```

```
        return

if args.validate:
    validate_model(args.validate, args.data, args.device)
    return

train_model(
    data_yaml=args.data,
    model_name=args.model,
    epochs=args.epochs,
    batch_size=args.batch,
    img_size=args.imgsz,
    device=args.device,
    project=args.project,
    name=args.name,
    resume=args.resume,
    patience=args.patience,
)

if __name__ == "__main__":
    main()
```

```
=====
tests/test_detector.py | Unit tests - 25 tests covering config, preprocessing, size, severity
=====
```

```
"""
```

```
Unit Tests for Road Pothole Detection System
```

```
=====
```

```
Run with: pytest tests/ -v
```

```
Tests cover:
```

- Configuration validation
- Preprocessing functions
- Size estimation
- Severity classification
- Utility functions (bbox extraction, IoU, etc.)

```
"""
```

```
import os
import sys
import json
import numpy as np
import cv2
import pytest

# Add project root to path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
import config
from src.utils import calculate_iou, generate_report_json
from src.size_estimator import SizeEstimator
from src.severity import SeverityClassifier
from src.preprocessing import enhance_contrast, reduce_noise, normalize_image
```

```
class TestConfig:
    """Test configuration constants."""

    def test_config_values_exist(self):
        assert hasattr(config, "CONFIDENCE_THRESHOLD")
        assert hasattr(config, "IOU_THRESHOLD")
        assert hasattr(config, "CLASS_NAMES")
        assert hasattr(config, "SIZE_SMALL_MAX_CM2")
        assert hasattr(config, "SIZE_MEDIUM_MAX_CM2")
        assert hasattr(config, "DEPTH_SHALLOW_MAX_CM")
        assert hasattr(config, "DEPTH_MODERATE_MAX_CM")

    def test_confidence_threshold_range(self):
        assert 0.0 < config.CONFIDENCE_THRESHOLD < 1.0

    def test_size_thresholds_ordering(self):
        assert config.SIZE_SMALL_MAX_CM2 < config.SIZE_MEDIUM_MAX_CM2

    def test_depth_thresholds_ordering(self):
        assert config.DEPTH_SHALLOW_MAX_CM < config.DEPTH_MODERATE_MAX_CM

    def test_severity_levels_exist(self):
        for level in ["Critical", "High", "Medium", "Low"]:
            assert level in config.SEVERITY_LEVELS
```

```
class TestPreprocessing:
    """Test image preprocessing functions."""

    @pytest.fixture
    def sample_image(self):
        """Create a sample image for testing."""
        img = np.random.randint(0, 255, (100, 100, 3), dtype=np.uint8)
        return img

    def test_enhance_contrast(self, sample_image):
        enhanced = enhance_contrast(sample_image)
        assert enhanced.shape == sample_image.shape
        assert enhanced.dtype == sample_image.dtype

    def test_reduce_noise(self, sample_image):
        denoised = reduce_noise(sample_image)
        assert denoised.shape == sample_image.shape
        assert denoised.dtype == sample_image.dtype

    def test_normalize_image(self, sample_image):
        normalized = normalize_image(sample_image)
        assert normalized.shape == sample_image.shape
```

```

        assert normalized.dtype == np.float32

    def test_preprocess_preserves_shape(self, sample_image):
        from src.preprocessing import preprocess_image
        result = preprocess_image(sample_image, normalize=False)
        assert result.shape == sample_image.shape

class TestSizeEstimator:
    """Test size estimation logic."""

    @pytest.fixture
    def estimator(self):
        return SizeEstimator()

    def test_size_classification_small(self, estimator):
        result = estimator.estimate_size((0, 0, 20, 20), 640)
        assert result.size_category in ["Small", "Unknown"]

    def test_size_classification_medium(self, estimator):
        # Create a bbox that should produce medium size area
        result = estimator.estimate_size((0, 0, 200, 200), 640)
        assert result.size_category in ["Small", "Medium", "Large"]

    def test_size_calculation_positive(self, estimator):
        result = estimator.estimate_size((0, 0, 100, 100), 640)
        assert result.width_cm > 0
        assert result.height_cm > 0
        assert result.area_cm2 > 0

    def test_size_result_attributes(self, estimator):
        result = estimator.estimate_size((10, 10, 110, 110), 640)
        assert hasattr(result, "width_cm")
        assert hasattr(result, "height_cm")
        assert hasattr(result, "area_cm2")
        assert hasattr(result, "size_category")
        assert hasattr(result, "pixel_density_cm_per_px")

class TestSeverityClassifier:
    """Test severity classification logic."""

    @pytest.fixture
    def classifier(self):
        return SeverityClassifier()

    def test_low_severity_small_shallow(self, classifier):
        result = classifier.classify(
            area_cm2=100, size_category="Small",
            depth_cm=1.0, depth_category="Shallow"
        )
        assert result.severity == "Low"
        assert result.priority == 4

    def test_critical_severity_large_deep(self, classifier):
        result = classifier.classify(
            area_cm2=5000, size_category="Large",
            depth_cm=8.0, depth_category="Deep"
        )
        assert result.severity == "Critical"
        assert result.priority == 1

    def test_medium_severity_medium_moderate(self, classifier):
        result = classifier.classify(
            area_cm2=800, size_category="Medium",
            depth_cm=3.5, depth_category="Moderate"
        )
        assert result.severity in ["Medium", "High"]

    def test_severity_result_has_all_fields(self, classifier):
        result = classifier.classify(
            area_cm2=500, size_category="Medium",
            depth_cm=2.5, depth_category="Moderate"
        )
        assert hasattr(result, "severity")
        assert hasattr(result, "priority")
        assert hasattr(result, "color_bgr")
        assert hasattr(result, "response_time_hours")
        assert hasattr(result, "recommendation")

    def test_matrix_classification(self, classifier):
        result = classifier.classify_matrix("Large", "Deep")

```

```

        assert result.severity == "Critical"

def test_recommendation_generated(self, classifier):
    result = classifier.classify(
        area_cm2=5000, size_category="Large",
        depth_cm=10.0, depth_category="Deep"
    )
    assert len(result.recommendation) > 0
    assert "Immediate" in result.recommendation or "repair" in result.recommendation.lower()

class TestUtils:
    """Test utility functions."""

    def test_calculate_iou_perfect_overlap(self):
        box1 = [0, 0, 100, 100]
        box2 = [0, 0, 100, 100]
        assert calculate_iou(box1, box2) == 1.0

    def test_calculate_iou_no_overlap(self):
        box1 = [0, 0, 10, 10]
        box2 = [100, 100, 200, 200]
        assert calculate_iou(box1, box2) == 0.0

    def test_calculate_iou_partial_overlap(self):
        box1 = [0, 0, 100, 100]
        box2 = [50, 50, 150, 150]
        iou = calculate_iou(box1, box2)
        assert 0.0 < iou < 1.0

    def test_generate_report_structure(self):
        detections = [
            {"bbox": [10, 20, 50, 60], "confidence": 0.95,
             "severity": "High", "size_cm2": 800, "size_category": "Medium"},
            {"bbox": [100, 200, 150, 250], "confidence": 0.85,
             "severity": "Low", "size_cm2": 200, "size_category": "Small"},
        ]
        report = generate_report_json(detections, "test.jpg", (37.7749, -122.4194))
        assert "timestamp" in report
        assert "gps_coordinates" in report
        assert "total_potholes_detected" in report
        assert report["total_potholes_detected"] == 2
        assert report["summary"]["high_count"] == 1
        assert report["summary"]["low_count"] == 1

class TestDetectionResult:
    """Test DetectionResult dataclass."""

    def test_detection_to_dict(self):
        import sys
        sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
        from src.detector import DetectionResult

        det = DetectionResult(
            bbox=[10, 20, 100, 200],
            confidence=0.95,
            class_id=0,
            class_name="Pothole",
            width_px=90,
            height_px=180,
            center=(55, 110),
            size_cm2=750.0,
            size_category="Medium",
            width_cm=15.5,
            height_cm=31.0,
            depth_cm=3.5,
            depth_category="Moderate",
            severity="High",
            priority=2,
            response_time_hours=168,
        )

        d = det.to_dict()
        assert d["class_name"] == "Pothole"
        assert d["confidence"] == 0.95
        assert d["severity"] == "High"
        assert d["priority"] == 2

class TestIntegration:
    """Integration test for the full pipeline (requires torch)."""

```

```

@pytest.fixture
def real_image(self):
    """Load a sample road image if available."""
    sample_paths = [
        os.path.join(config.SAMPLES_DIR, "road1.jpg"),
        os.path.join(config.SAMPLES_DIR, "road2.jpg"),
        os.path.join(config.SAMPLES_DIR, "sample.jpg"),
    ]
    for path in sample_paths:
        if os.path.exists(path):
            return cv2.imread(path)
    # Create a dummy image if no sample exists
    return np.random.randint(0, 255, (480, 640, 3), dtype=np.uint8)

@pytest.mark.skipif(not os.path.exists(config.YOLO_MODEL_PATH),
                    reason="YOLO model not available")
def test_full_pipeline(self, real_image):
    """Test the complete detection pipeline."""
    from main import PotholeAnalysisPipeline

    pipeline = PotholeAnalysisPipeline()
    result = pipeline.analyze_image(real_image)

    assert "detections" in result
    assert "report" in result
    assert "processing_time_seconds" in result
    assert "annotated_image" in result

if __name__ == "__main__":
    pytest.main([__file__, "-v", "--tb=short"])

```